

A person in a dark suit stands with their back to the camera, looking out from a dark, rocky cave opening. The cave's irregular shape frames a panoramic view of a dense city skyline, with the Freedom Tower prominently visible in the center. The city is bathed in a soft, hazy light, suggesting dawn or dusk. The overall mood is contemplative and visionary.

SMILE ARCHITECTURE

Workshop – 3 Mar 2021



CENTER FOR DEVELOPMENT OF
ADVANCED COMPUTING



Agenda

01 Company Profile

About ECGC

02 Business Functions

Details on business modules

03 System Architecture

Proposed Architecture for SMILE

04 Decision

Conclude on choice of Architecture

Detailed Agenda



ABOUT ECGC



STAKEHOLDERS



JOURNEY



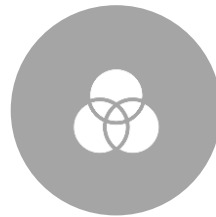
PAIN POINTS



DRIVERS



ARCHITECTURES



COMPARISON



EVAL. & RECO.



@ ECGC



(A Government of India Enterprise)

You focus on exports. We cover the risks.

About ECGC



DOMAIN



INSURANCE

SUB-DOMAIN

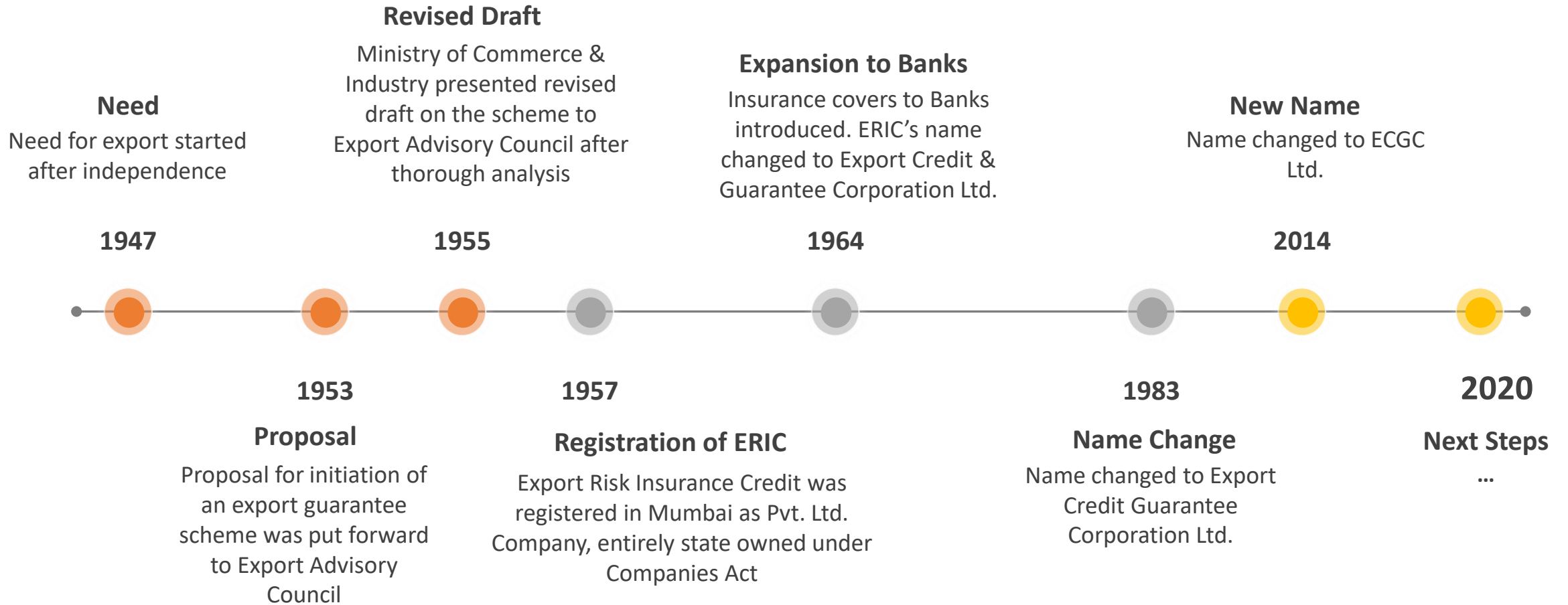


EXPORT CREDIT INSURANCE

About ECGC



Timeline



Vision and Mission



To excel in providing export credit insurance and trade-related services.



To support the Indian export industry by providing cost-effective insurance and trade-related services to meet the growing needs of the Indian export market through the optimal utilisation of available resources.

Objectives



1

To encourage and facilitate globalization of India's trade

2

To assist Indian exporters in managing their credit risks by providing timely information on worthiness of the buyers, bankers and the countries

3

To protect the Indian exporters against unforeseen losses, which may arise due to failure of the buyer, bank or problems faced by the country of the buyer by providing cost effective credit insurance covers in the form of Policy, Factoring and Investment Insurance Services comparable to similar covers available to exporters in other countries

4

To facilitate availability of adequate bank finance to the Indian exporters by providing surety insurance covers for bankers at competitive rates

5

To achieve improved performance in terms of profitability, financial and operational efficiency indicators and achieve optimum return on investment

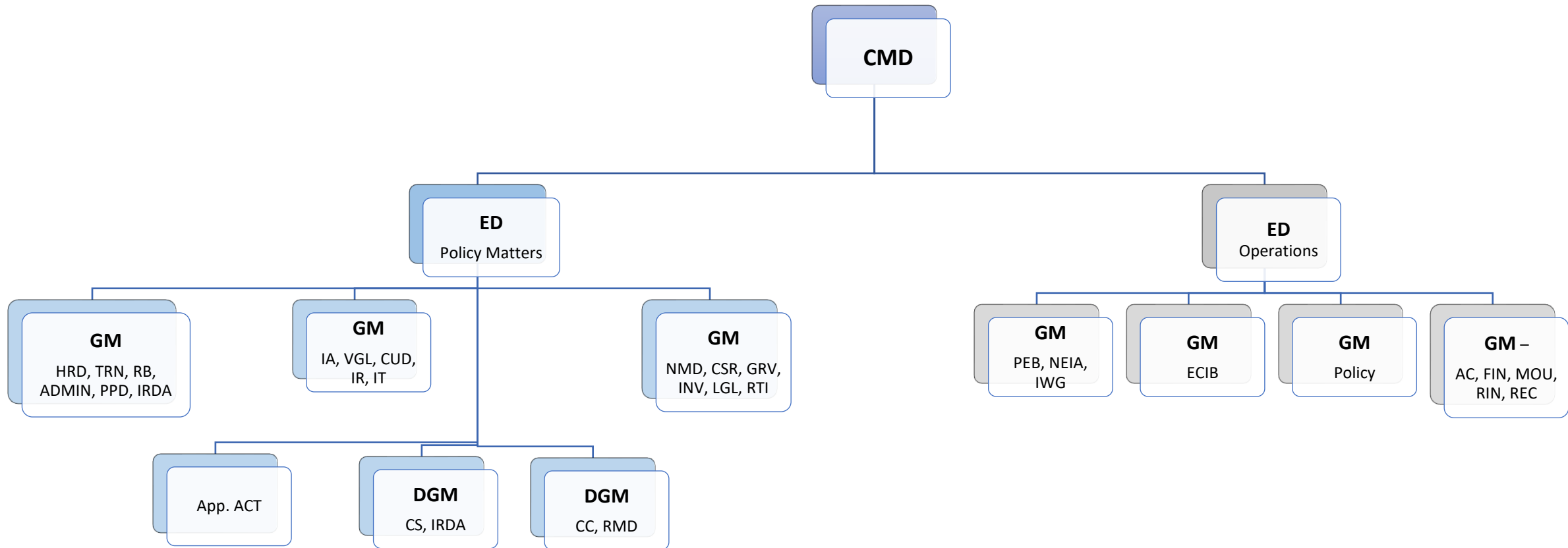
6

To develop world class expertise in credit insurance among employees and ensure continuous innovation and achieve the highest customer satisfaction by delivering top quality service

7

To educate the customers by continuous publicity and effective marketing

Organization



Centers

Western Region

Maharashtra
Dadra and Nagar Haveli
Daman and Diu
Goa
Gujarat
Madhya Pradesh (part)

Southern Region - 2

Karnataka
Andhra Pradesh
Kerala
Lakshadweep
Telangana

■ Mumbai HO

Western Region

Mumbai RO

GIFT City
Ahmedabad EB
Ahmedabad BBB
Rajkot
Vadodara
Indore
Surat
Thane
Andheri
Bandra BBB
Bandra SME
Mumbai BBB
Mumbai LEB
Mumbai PEB
Churchgate
Pune
Nagpur

Southern Region 2

Bangalore RO

Mangalore
Hyderabad BBB
Hyderabad EB
Bangalore EB
Bangalore BBB
Guntur
Kochi
Vishakhapatnam
Kollam

Northern Region

Delhi RO

Srinagar
Jalandhar
Chandigarh
Ludhiana
Panipat
Faridabad
Delhi LEB
Delhi BBB
Gurgaon
Noida
West Delhi
Delhi SME
Jaipur
Moradabad
Kanpur
Jodhpur
Agra

Northern Region

Delhi
Chandigarh
Haryana
Himachal Pradesh
Jammu and Kashmir
Jharkhand
Madhya Pradesh (part)
Punjab
Rajasthan
Uttar Pradesh (West)
Uttarakhand

Eastern Region

West Bengal
Bihar
Chhattisgarh
Odisha
Seven NE states
Sikkim
Uttar Pradesh (East)
Chattisgarh

Eastern Region

Kolkata RO

Guwahati (Sub-Office)
Kolkata BBB
Kolkata EB
Varanasi
Bhubaneswar
Raipur

Southern Region 1

Chennai RO

Periamet
Chennai BBB
Chennai EB
Chennai South
Salem
Karur
Tuticorin
Tirupur BBB
Tirupur EB
Coimbatore
Madurai

Southern Region - 1

Tamil Nadu
Andaman and Nicobar
Islands
Puducherry

Products & Services



- Offers insurance protection to exporters against payment risks
- Provides guidance in export-related activities
- Makes available information on different countries with its own credit ratings
- Makes it easy to obtain export finance from banks/financial institutions
- Assists exporters in recovering bad debts
- Provides information on credit-worthiness of overseas buyers

Products & Services

EXPORTERS

BANKS

SHORT TERM

ECIE - Turnover Based

1. Shipment Comprehensive Risk Policy (**SCR**)
2. Small Exporters Policy (**SEP**)
3. Specific Shipment Policy (**SSP**)
4. Services Policy (**SRC**)
5. Export Turnover Policy (**ETP**)
6. Exporter Specific Buyer Policy (**BWP**)
7. Consignment Exports Policy (**CSA**)

ECIE - Exposure Based

1. Buyer's Exposure Policy (**BEP**)
2. IT Enable Services Policy (**SITES**)
3. Micro Exporters Policy (**MEP**)
4. Software Project Policy (**SPP**)

ECIB – Pre-Shipment

1. Individual Packing Credit (**INPC**)
2. Whole Turnover Packing Credit (**WTPC**)
3. Branch Wise Packing Credit (**BIPC**)

ECIB – Post-Shipment

1. Individual Packing Credit (**INPS**)
2. ECIB-INPS (With Exclusion)
3. ECIB-INPS (Without Any Exclusion)
4. ECIB-INPS (Not Holding Standard Policy)
5. Whole Turnover Post Shipment (**WTPS**)
6. Export Finance (**EF**)

ECIB – Cover against Bank Guar.

1. Surety Cover (**SC**)
2. Advance Payment-cum-Performance (**APP**)

Special Schemes

1. Factoring
2. Buyer's Credit Cover (**BC**)
3. Lines of Credit Cover (**LC**)
4. Overseas Investment Insurance
5. Customer Specific Covers
6. NEIA

MEDIUM & LONG TERM

Exposure Based

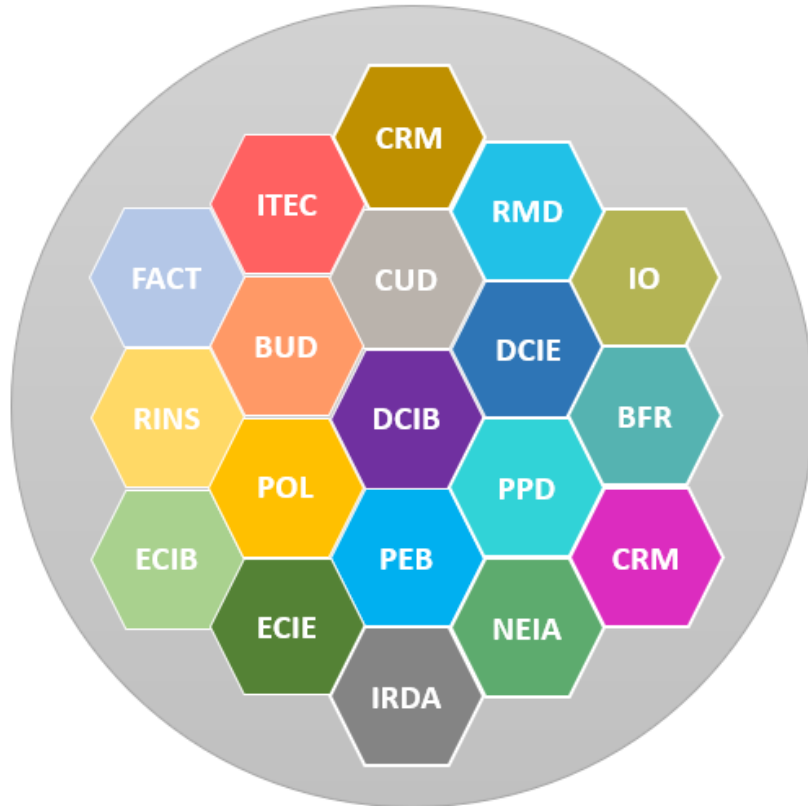
1. Construction Works Policy (**CWP**)
2. Specific Policy for Supply Contract (**SPSC**)
3. Specific Shipment Policy (**SSP**)
4. Specific Services Policy (**SRC**)
5. Letter of Credit confirmation Cover (**LCC**)

ECIB – MLT

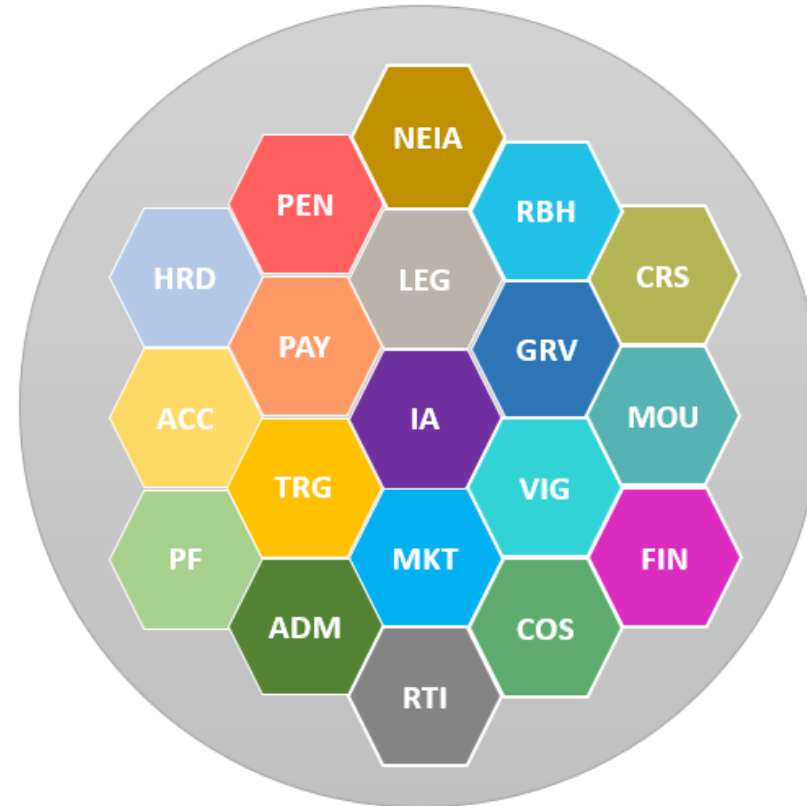
1. Individual Packing Credit (**INPC**)
2. Individual Post Shipment (**INPS**)
3. Export Performance (**EP**)
4. Export Finance[Overseas Lending] (**EF-OL**)
5. ECIB-Cash Flow Deficit Financing (**CFDF**)

Business Functions

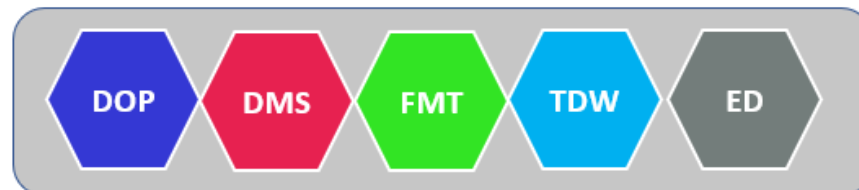
Core Business Modules



Support Modules



General Modules





Stakeholders

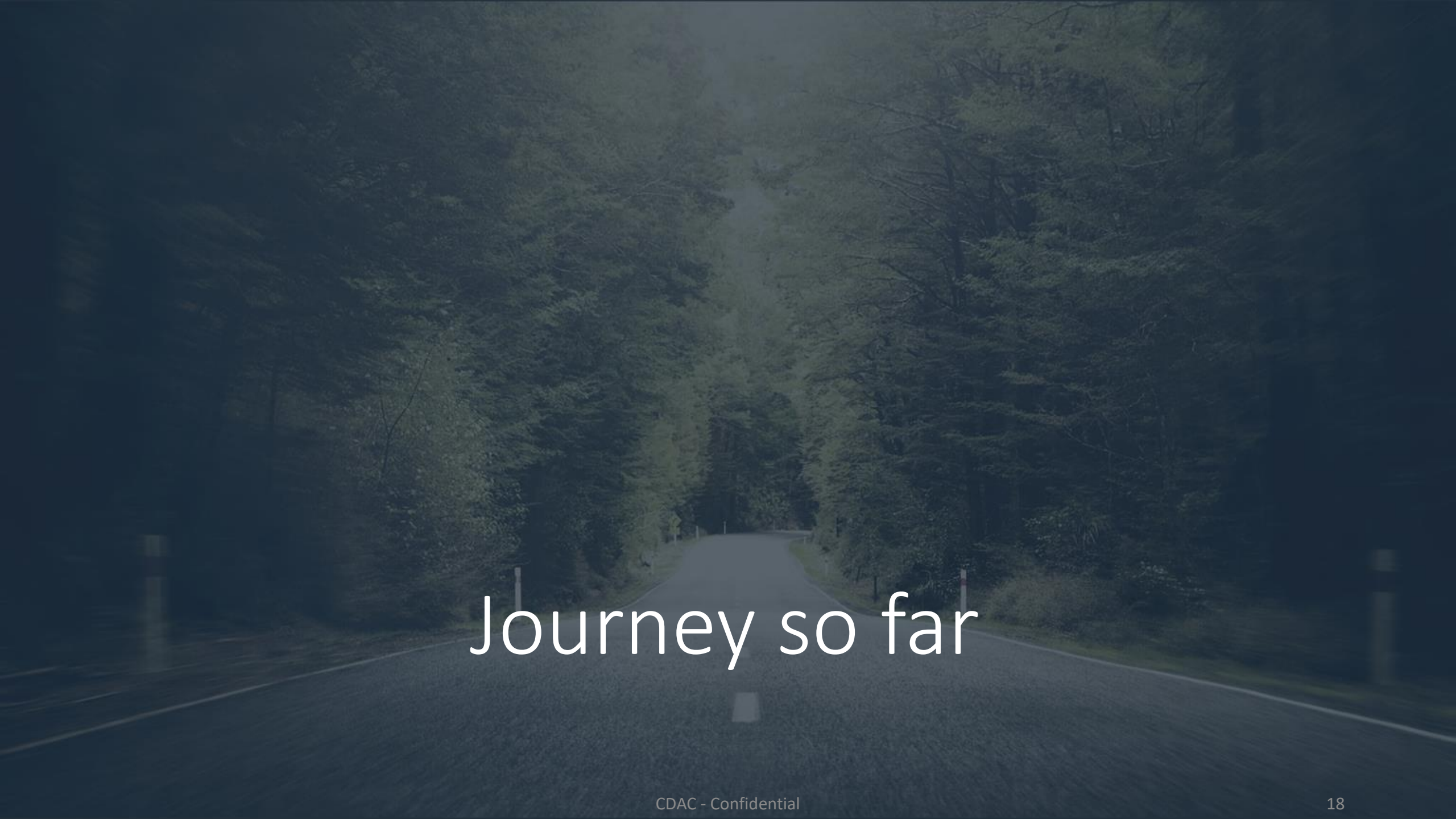
Stakeholders

DIRECT

- ECGC (Project Sponsor, Business, IT & Employees (Group A, B, C and D)
- Exporters
- Banks
- Brokers
- NGOs
- Reinsurer
- Internal Auditors
- IGMS
- Empowered Committees like HUC, HCC etc.
- C-DAC (Architecture and Design, Development, Testing & Maintenance)

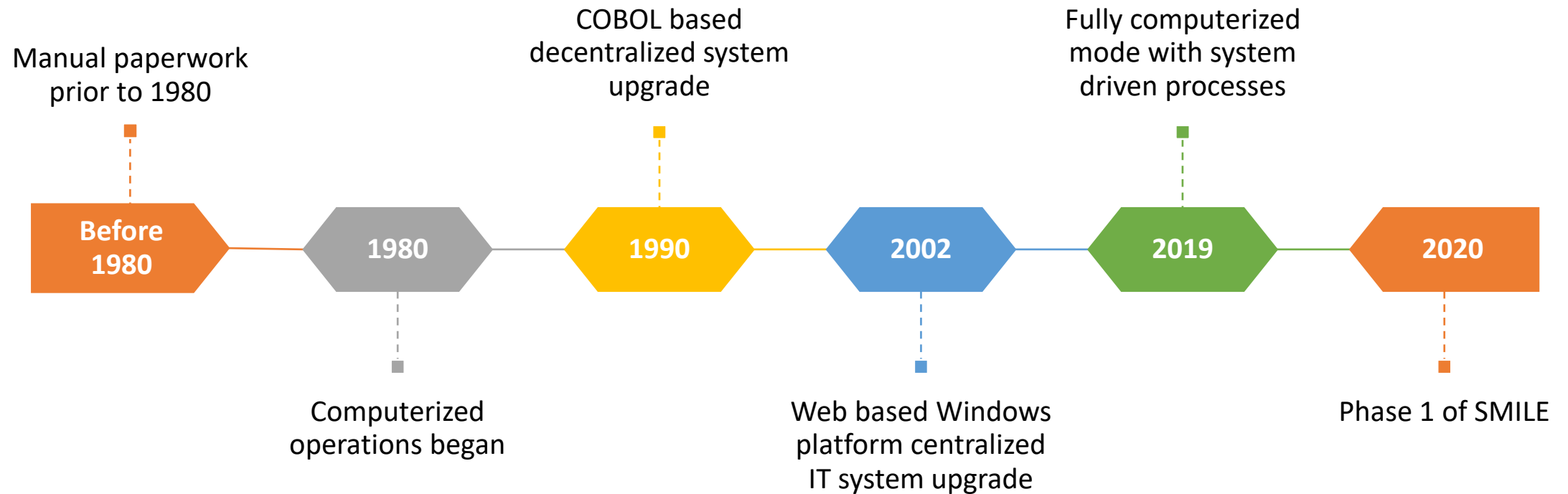
INDIRECT

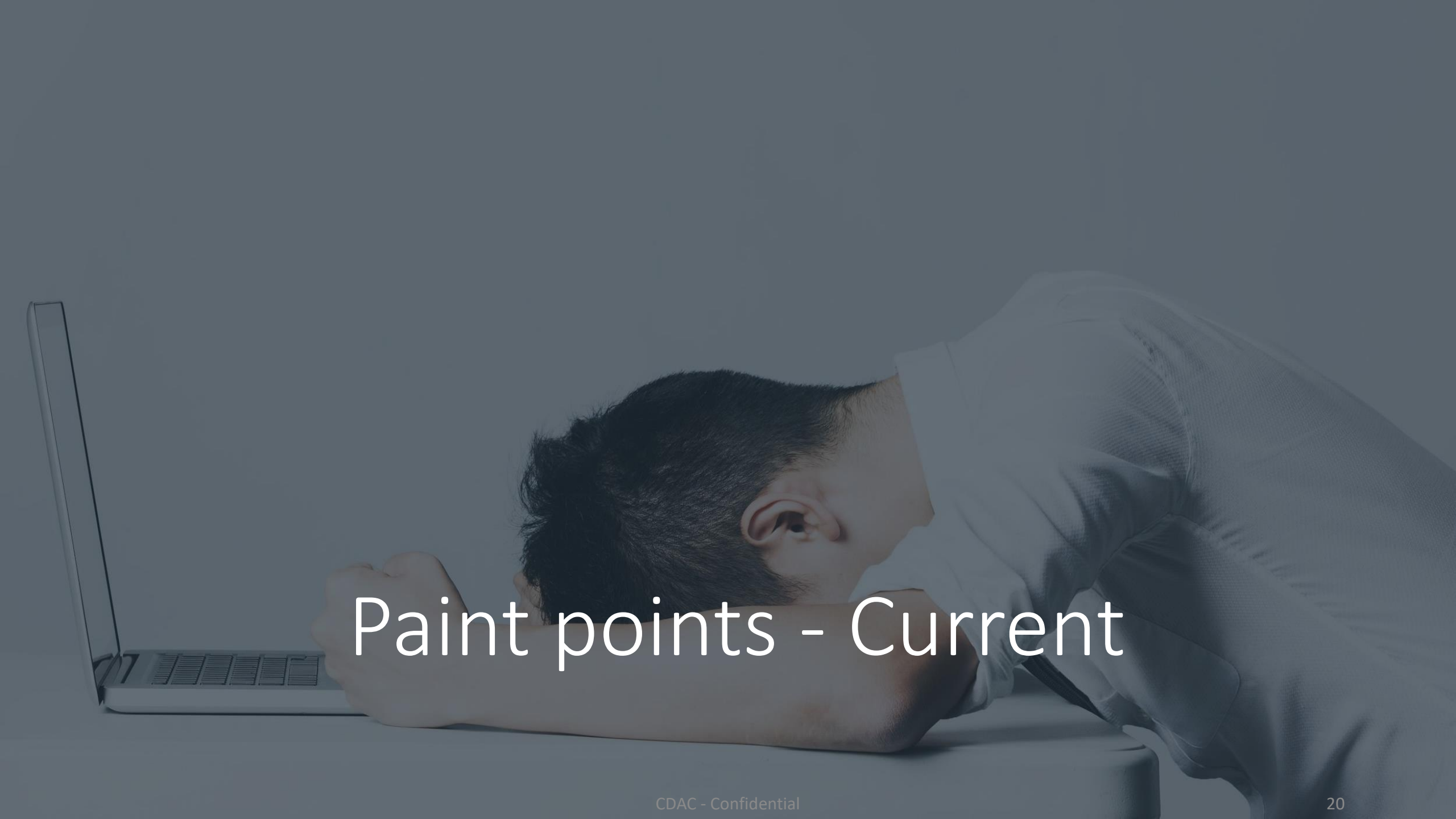
- Regulatory
 - IRDAI
 - RBI
 - I-T
 - PFRDA
 - CPGrams
 - External Auditors
 - TAC
- Ministry of Commerce
- Ministry of Home Affairs

A dark, atmospheric photograph of a road winding through a dense forest. The road is paved and leads into the distance, flanked by tall, leafy trees that create a canopy overhead. The lighting is dim, suggesting a misty or overcast day. The text "Journey so far" is overlaid in a large, white, sans-serif font in the center of the image.

Journey so far

Journey



A man in a white shirt is shown from the side, resting his head on his hand next to an open laptop. He appears to be tired or stressed. The background is a solid dark blue-grey color.

Paint points - Current

Pain Points – Current System



Technology

Built in ASP which is no longer supported

Non availability of Client Portal due to issues



Operational

Inability to respond quickly to the changes in Govt. policies (GST, IT, F16 etc.) or internal business processes (Filling of an inward)



Maintenance

Duplication of data
Report generation is difficult



Data security

Business data is exposed to IT
Confidential data not encrypted

A man in a dark suit stands with his back to the camera, looking out over a dense city skyline. The sky is overcast with grey clouds. The text "Drivers – Future System" is overlaid in white on the lower half of the image.

Drivers – Future System

Drivers – Future System



Business

~20% of business lost to other players



User experience

Non availability of online portal for conducting business



Regulatory

Fulfillment of regulatory requirements -
Reports needed by IRDAI,
Records of 10 years by IT
Data by Auditors etc.



Legal

Avoiding of issues like RTI filed, court cases due to portal issues



Technology

Technologies used for application no longer supported
Automation of workflow
Use of open source to avoid vendor lock-in
Support for multiple devices (Desktop, Laptop, Mobile – Phone/Tabs)



Operational

Efficiency of process
Increase in transparency
Paperless transactions

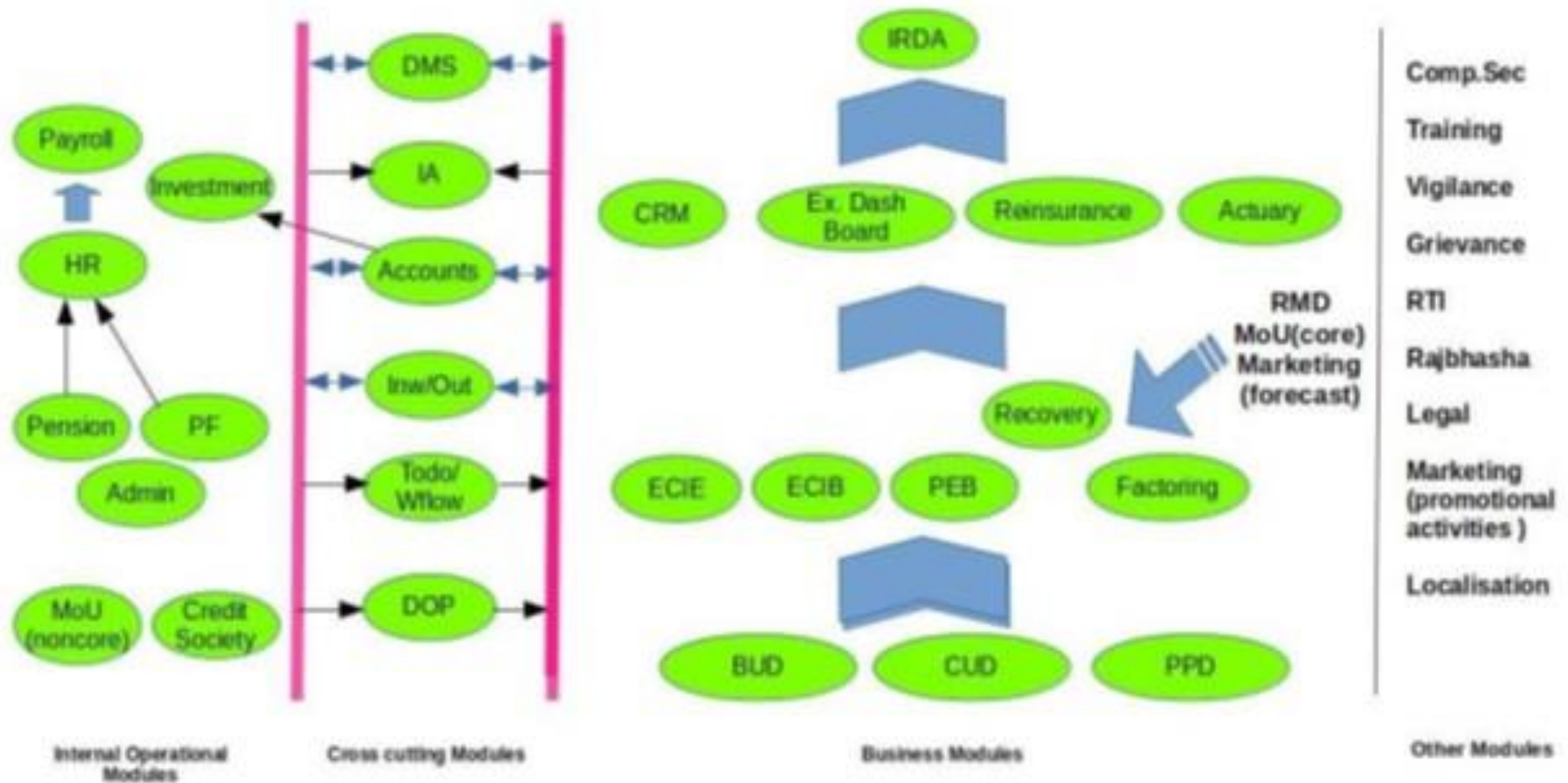
The background is a deep blue gradient with a faint, repeating pattern of binary code (0s and 1s) and a grid of hexagons. In the center, there is a 3D structure composed of translucent, light blue cubes arranged in a stepped, pyramid-like formation. The cubes have a slight glow and are semi-transparent, showing the grid lines underneath. The overall aesthetic is futuristic and technological.

Architectural Styles

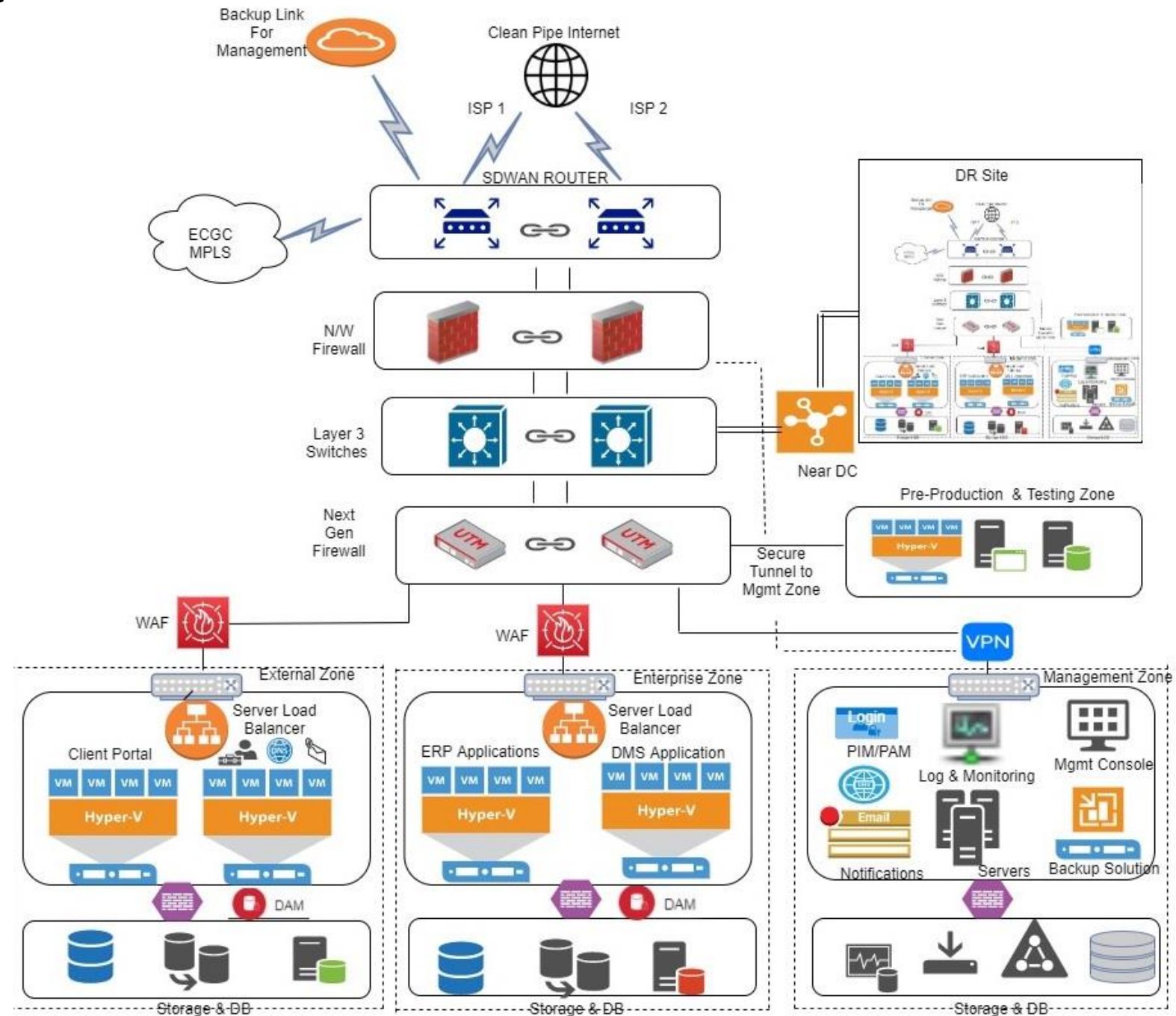
Architectural Styles

- **Monolithic Architecture (MA)**
- Service Oriented Architecture (SOA)
- **Micro Services Architecture (MSA)**
- Serverless Architecture (SA)

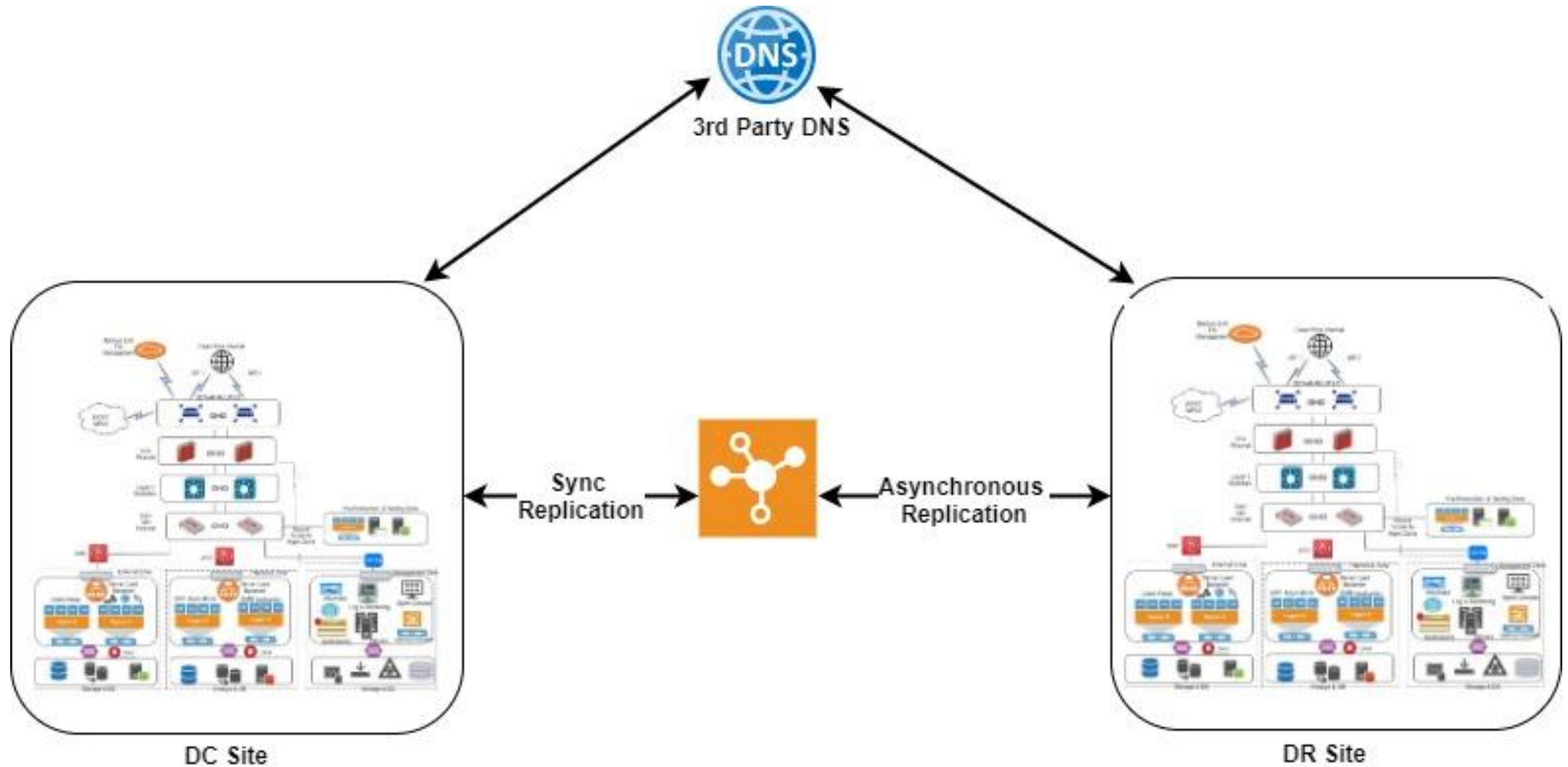
Business Architecture



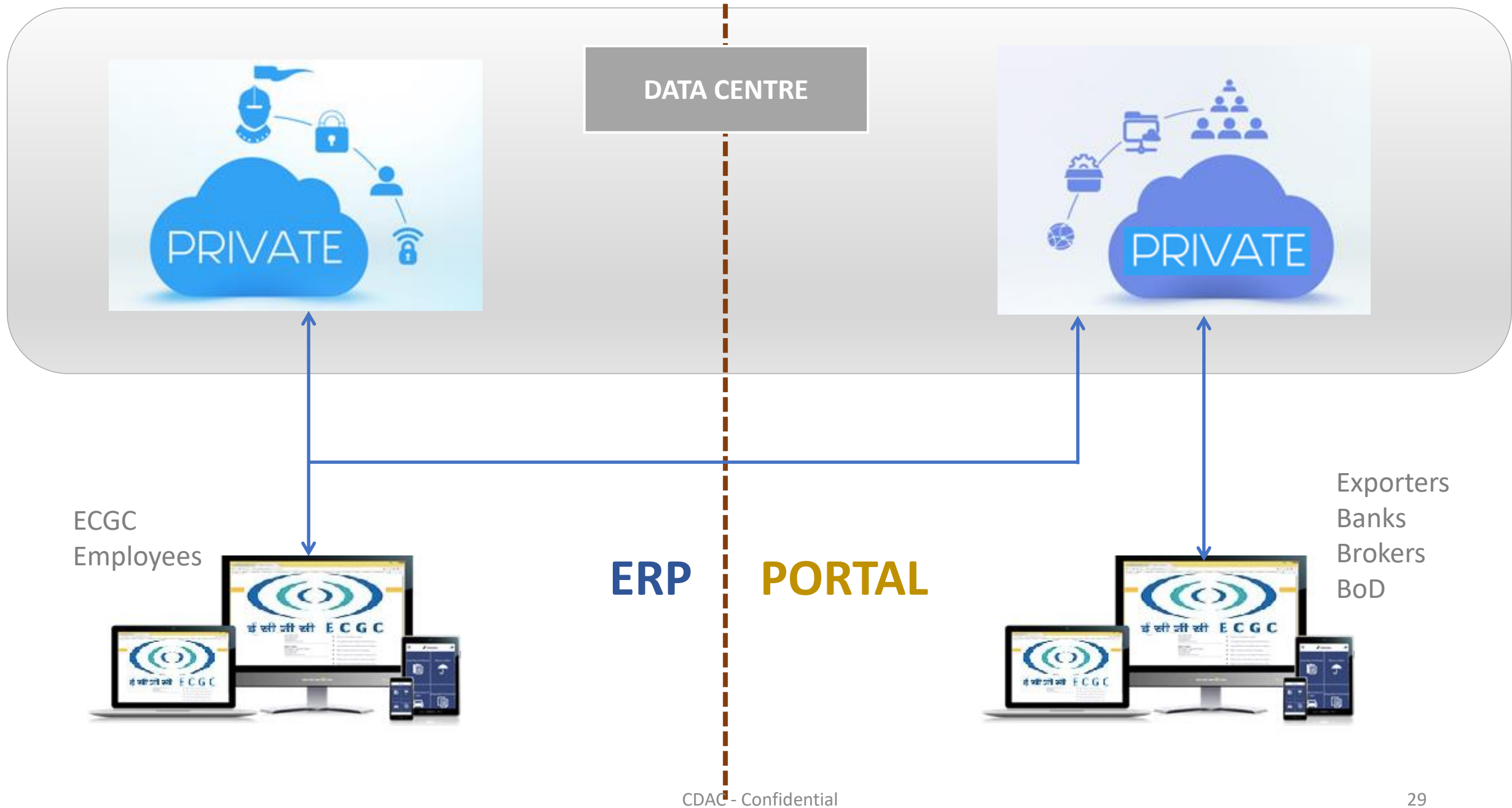
Physical Architecture



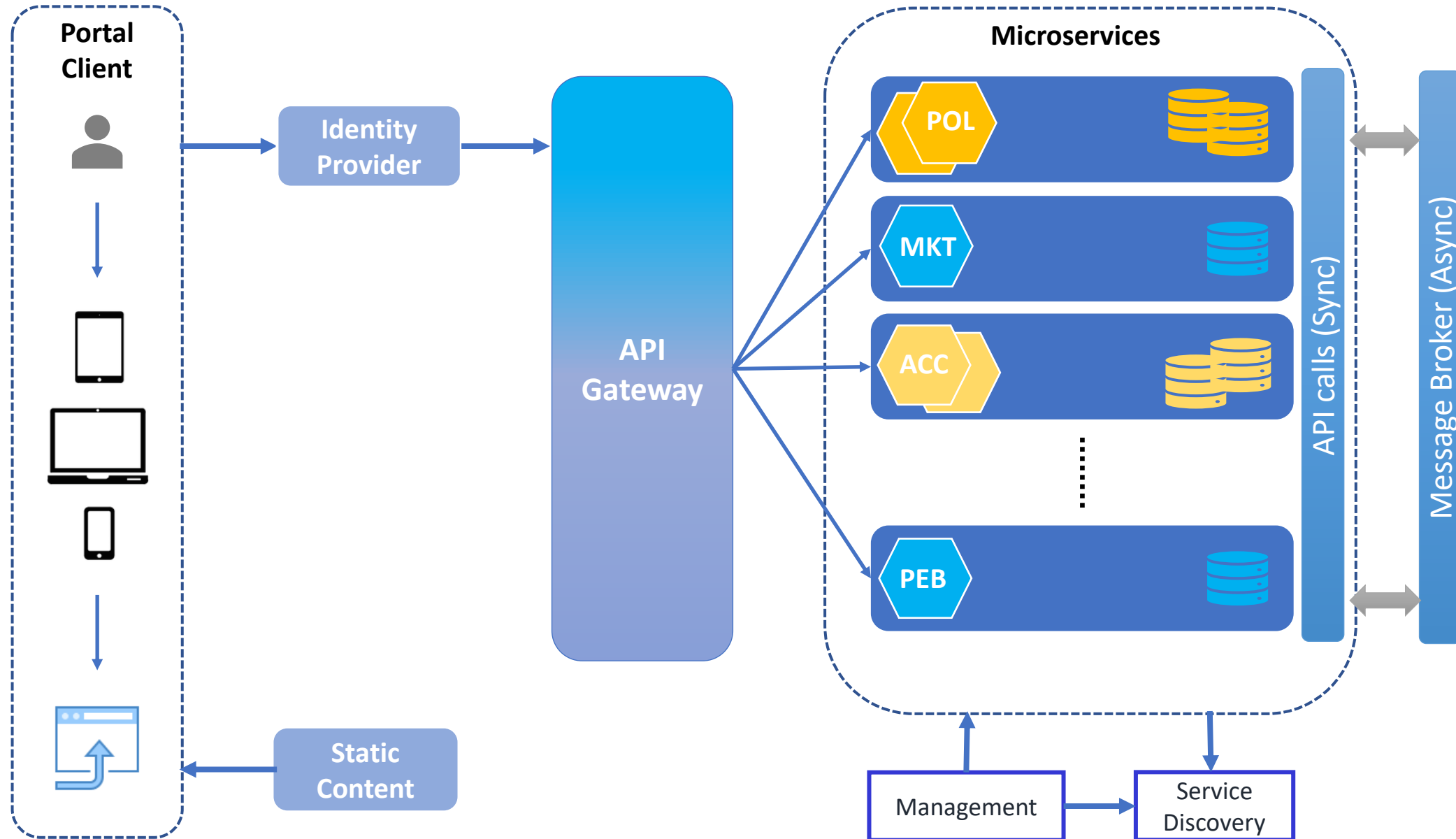
Physical Architecture - DC - DR



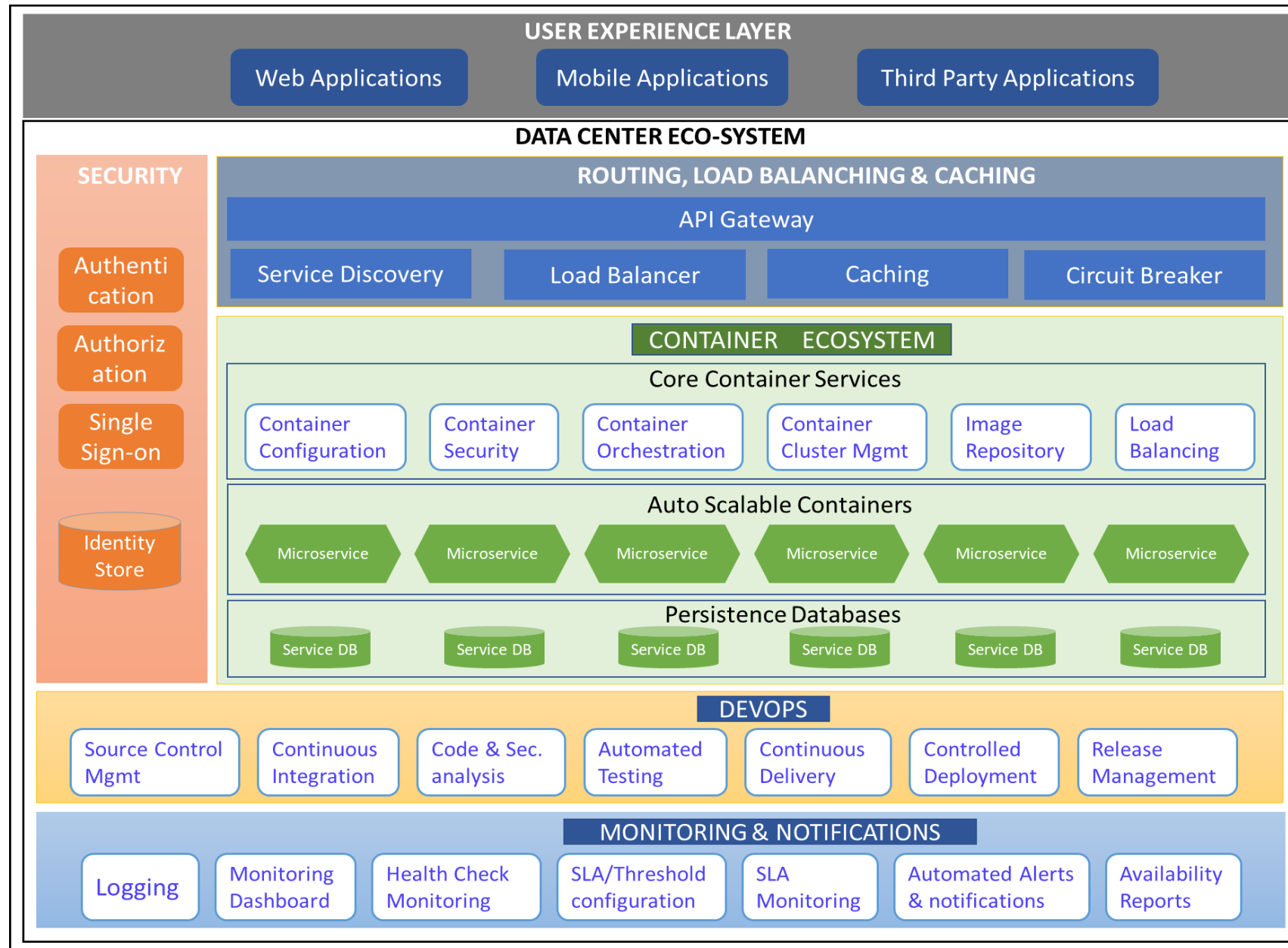
Logical View of Architecture

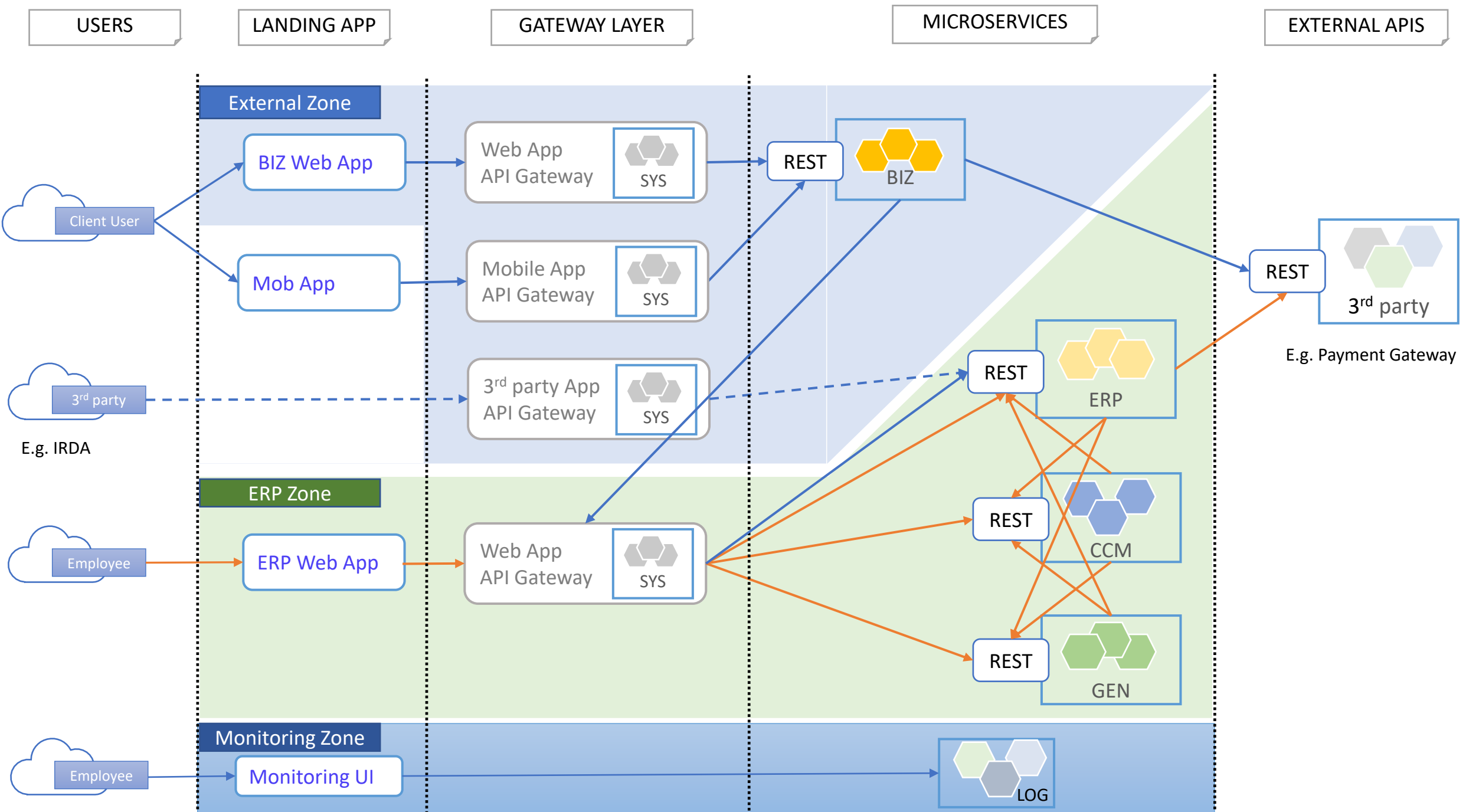


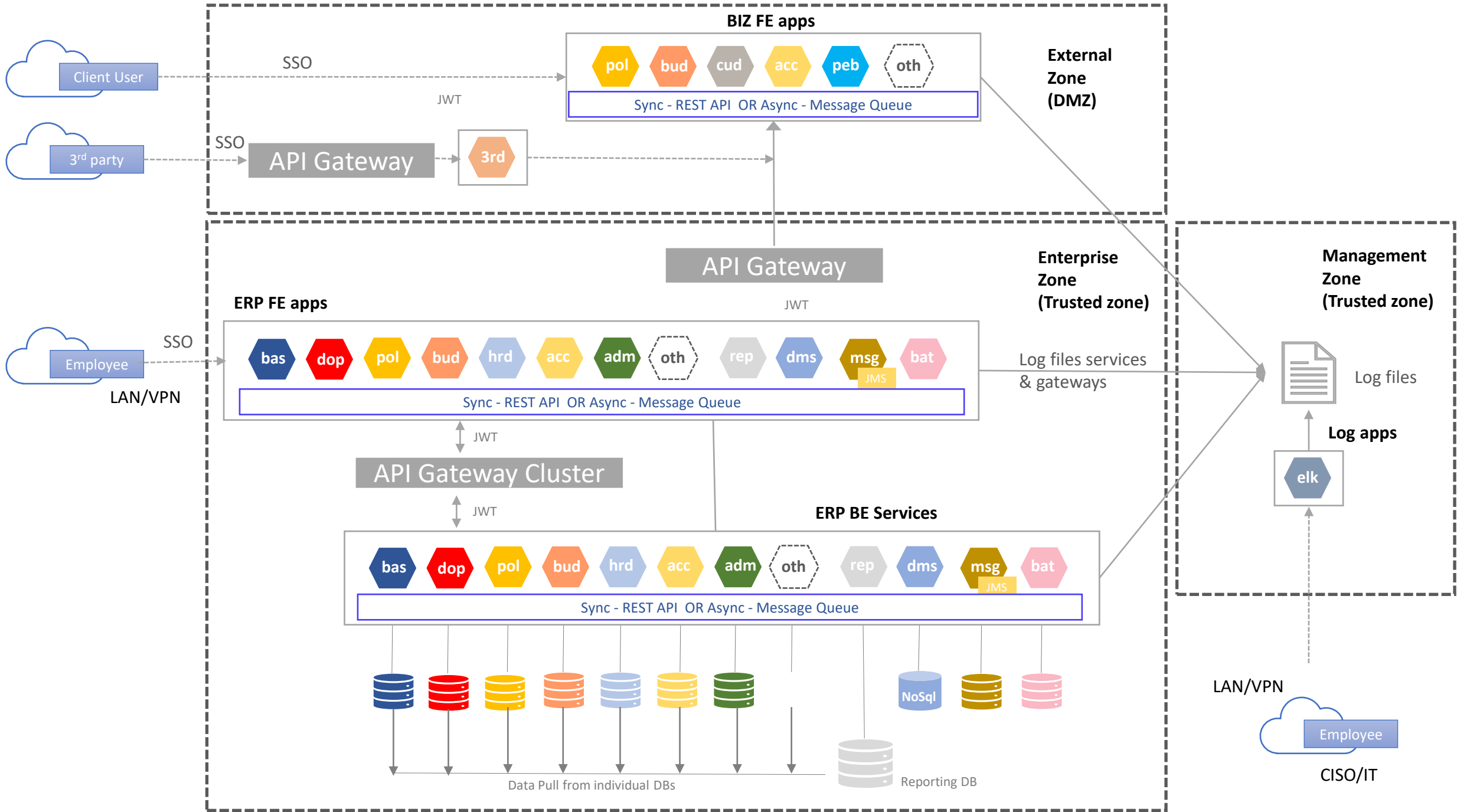
Logical Architecture



Reference Architecture









Welcome to ECGC

Welcome to the ECGC Client Portal, the online service interface for ECGC's customers. It provides various online services and enables exporters and bankers to seek information and submit various forms online.

Steps to Login:

1. Please select appropriate User Type (i.e. Bank or Exporter)
2. Please enter **User Id or IFSC Code** (For **Bank** type users) or **IE Code** (For **Exporter** type users)
3. Enter your password (**Virtual keyboard available** by ticking on checkbox below password field)
4. Type in the **Captcha Code** and click on Submit button
5. If login fails kindly check if **valid Policy or ECIB Cover** is available for the User Id / IFSC code / IE Code
6. **If you are an existing customer and logging in for the first time, kindly reset your password through [Forgot Password](#) to validate the credentials on new portal.**
7. If you are a new customer kindly register through [Sign Up](#)
8. Please contact your dealing branch [here](#) for **queries regarding/updation of Registered Email Id**

Login

User Type

☒ I am an Exporter☐ I am a Banker

Please sign-in with 10-digit IE code or 11-digit User Id already provided to you. Users logging in this new portal for the first time are requested to first change their password before logging in by clicking the "Forgot Password" link

User Id or IE Code

Password

☐ Enable Virtual Keyboard

What is the sum of

**14 + 9 =**[Forgot Password?](#)[Sign Up](#)



ECGC Limited

(A Government of India Enterprise)

Search

Helpline No. Login हिन्दी

- Home
- About Us
- Products & Services
- Service Network
- Customer Care
- Media Room
- RTI
- Branch Locator

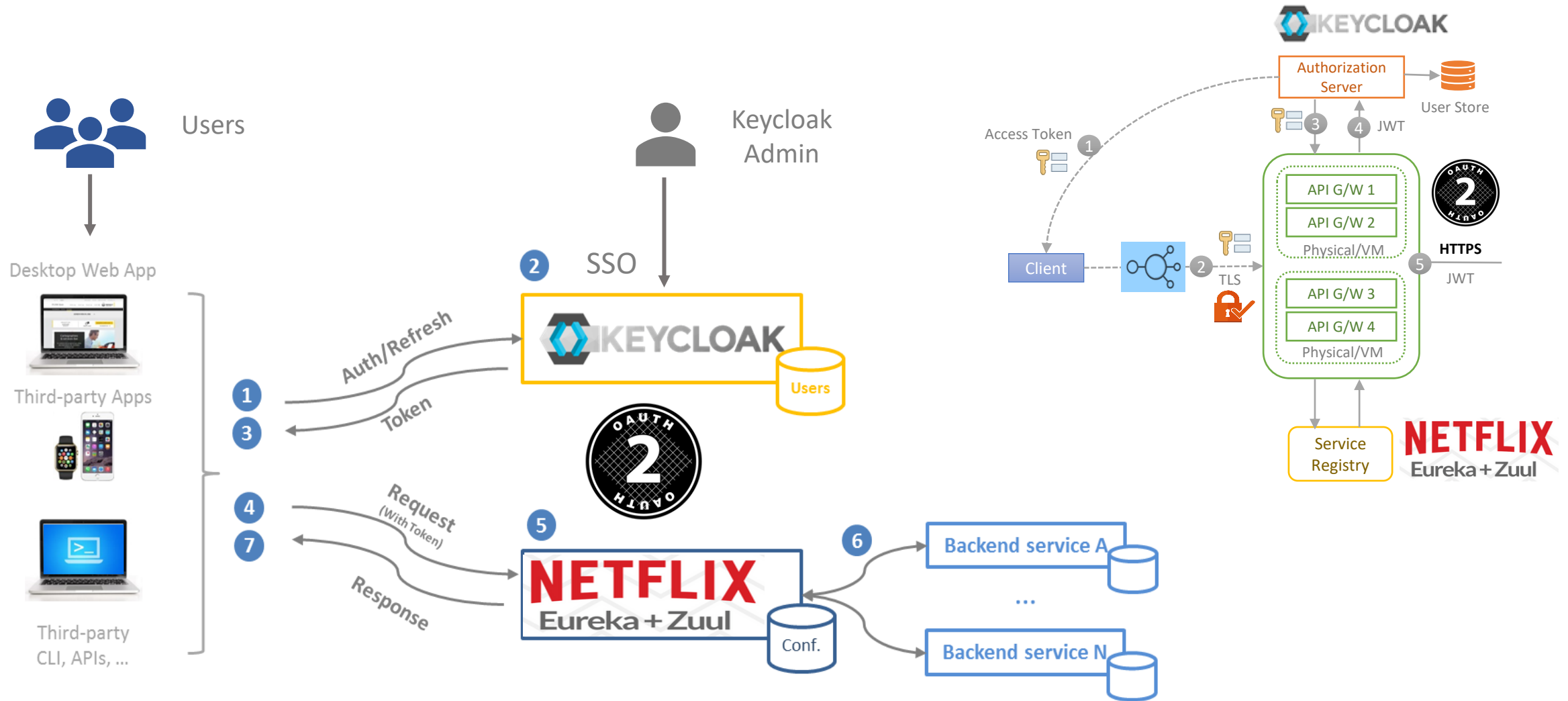
Announcements

NEW! Relaxations in time line for comp

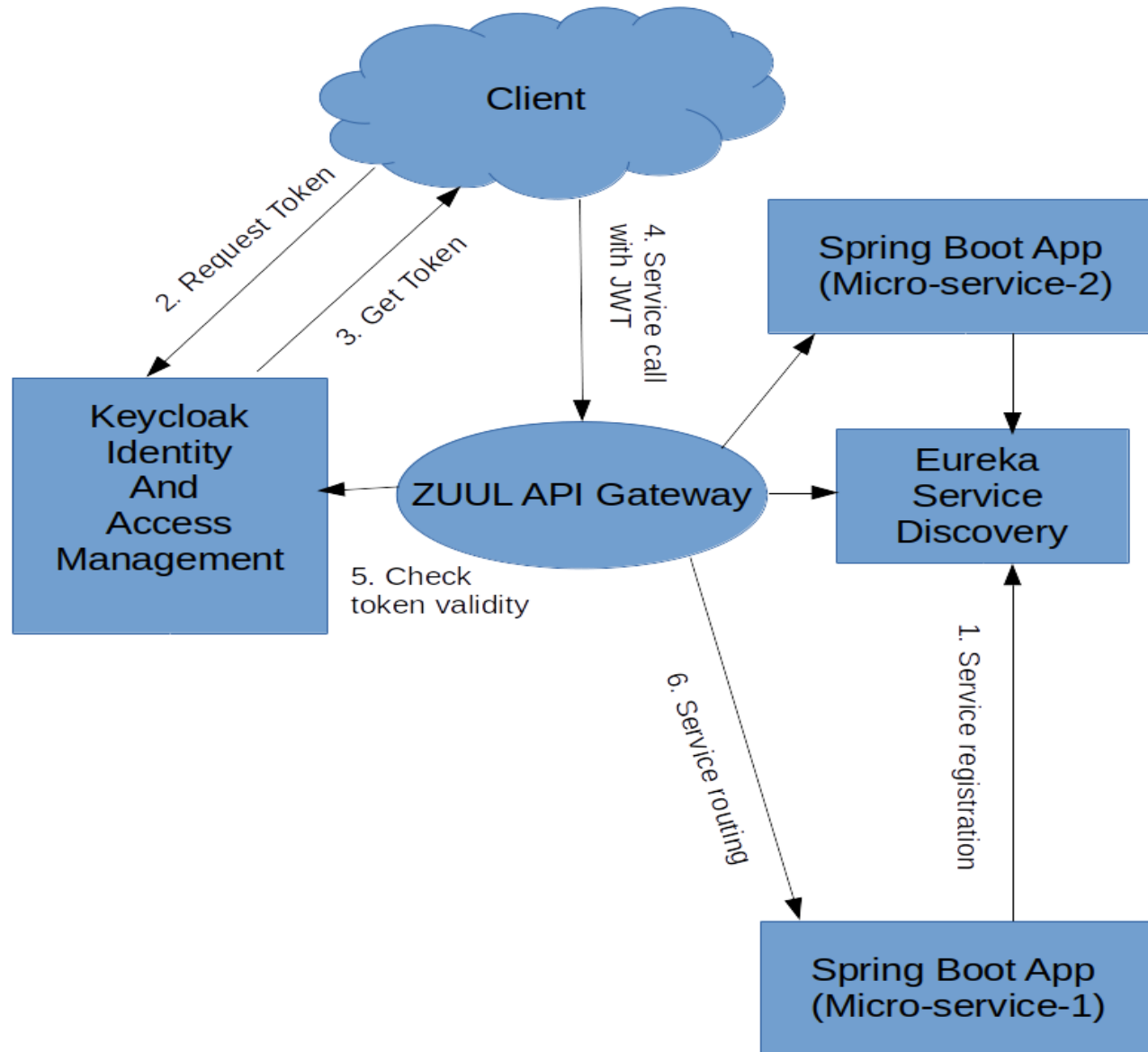


- Customer Grievance
- Public Disclosures
- RTI
- Audited Results
- Holiday List
- Branch Locator
- Awards & Recognitions
- Career With ECGC
- Tenders called for
- Product Bonds

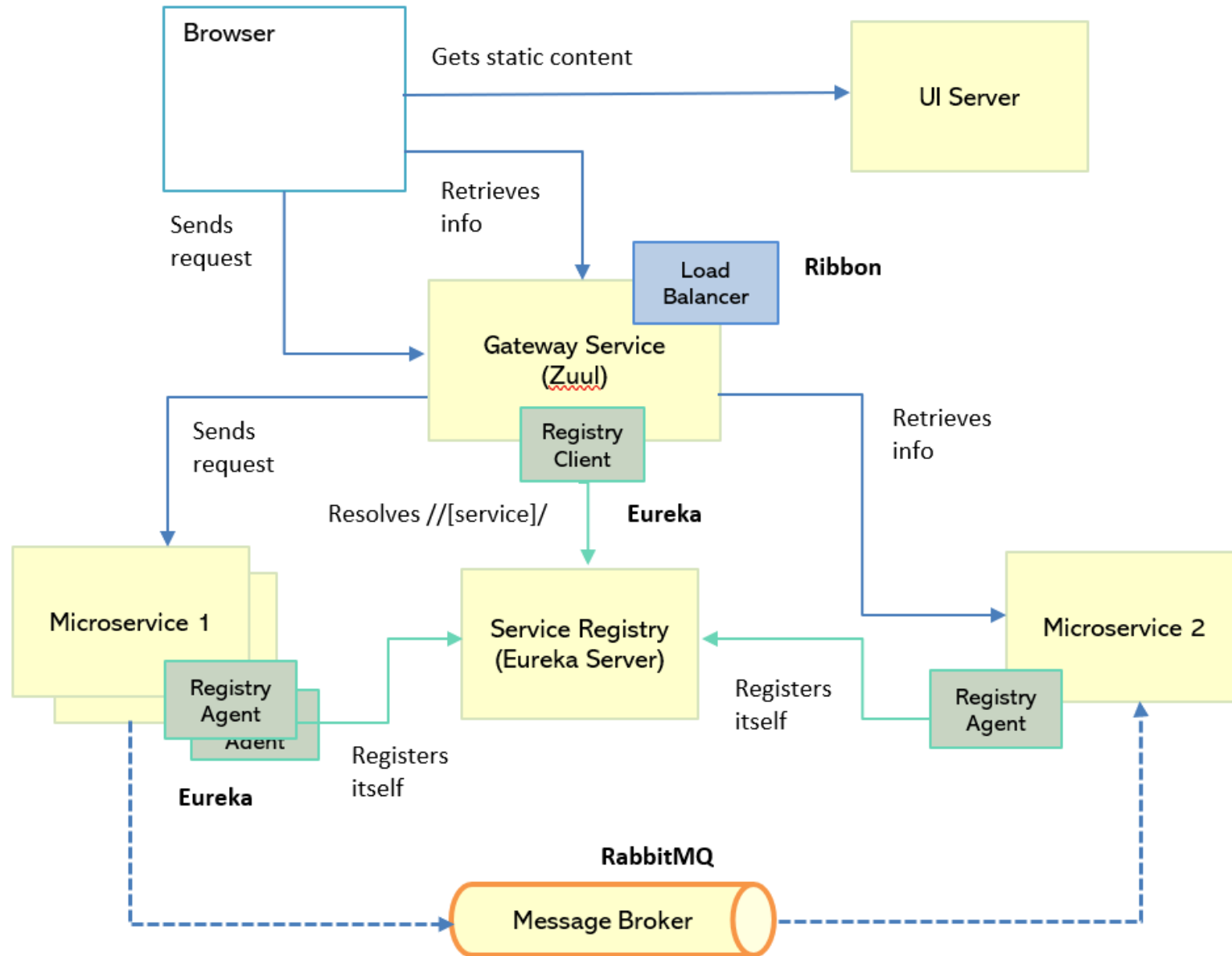
Authentication Architecture



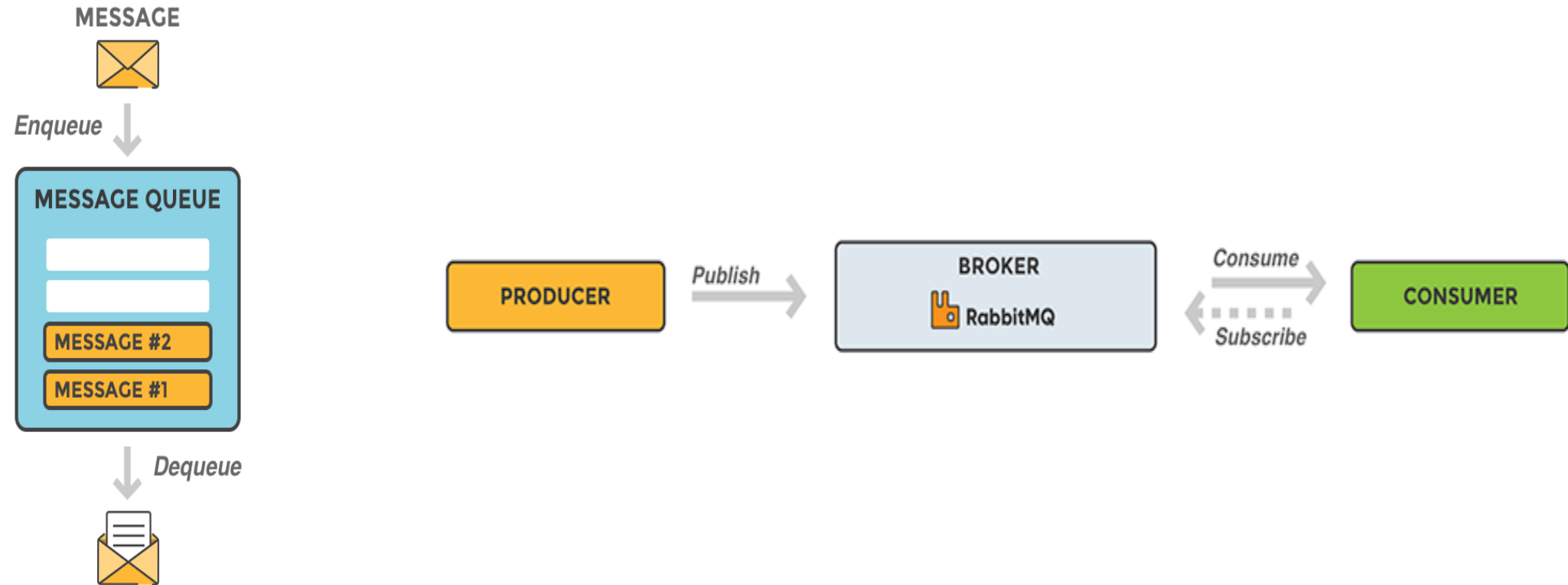
Authentication Architecture



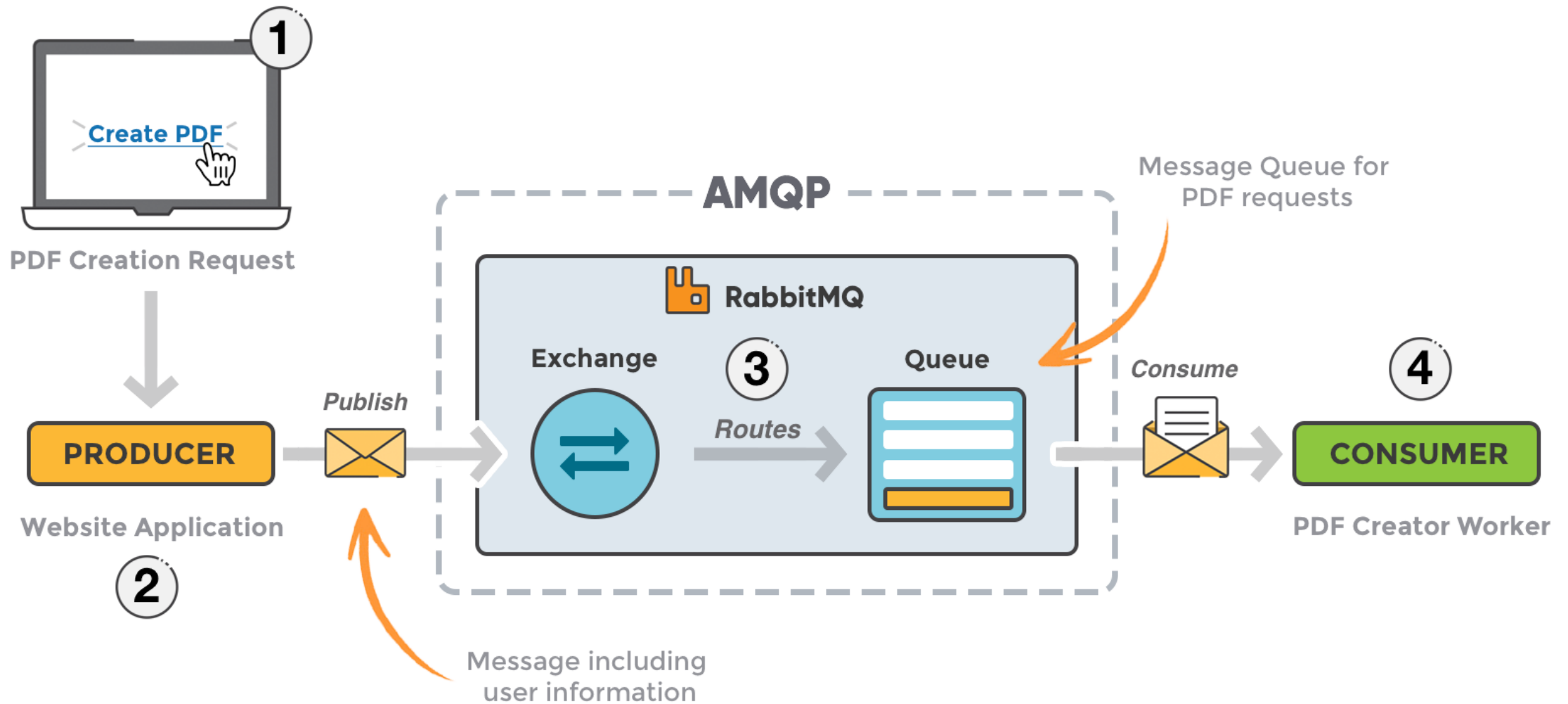
Service Discovery, Registry and Load Balancing



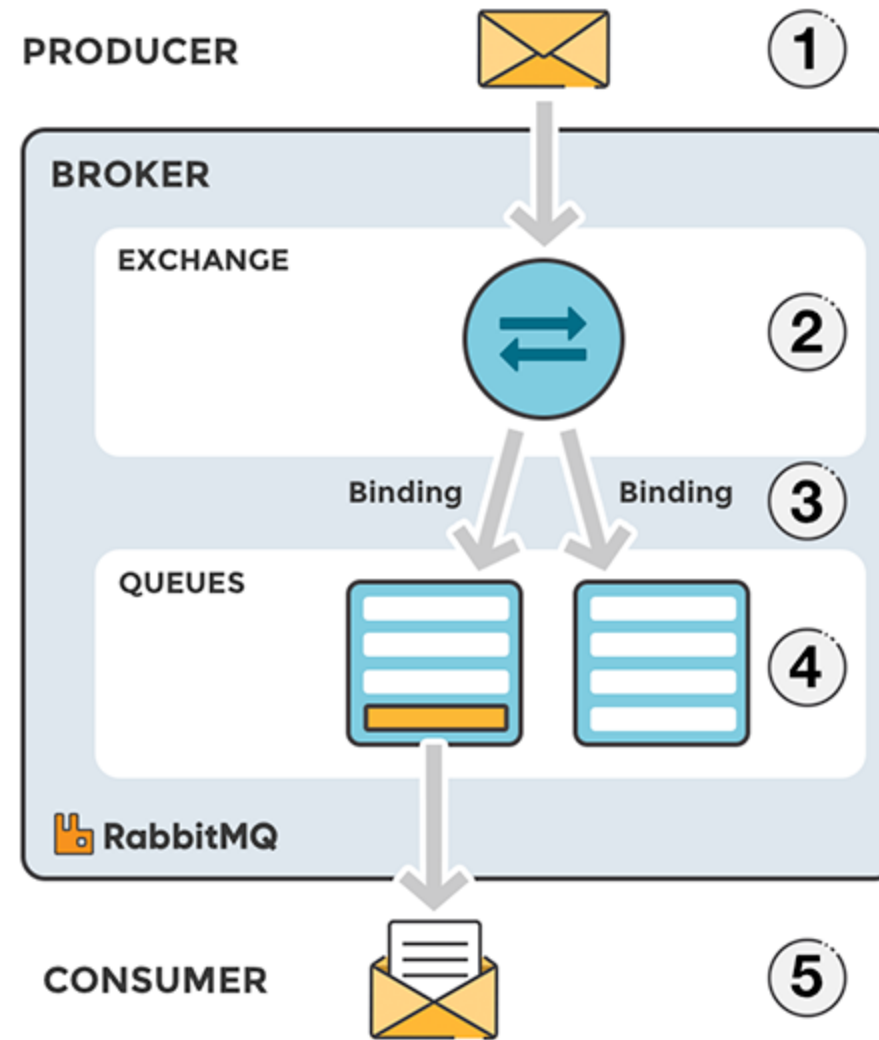
Communication between microservices - Asynchronous



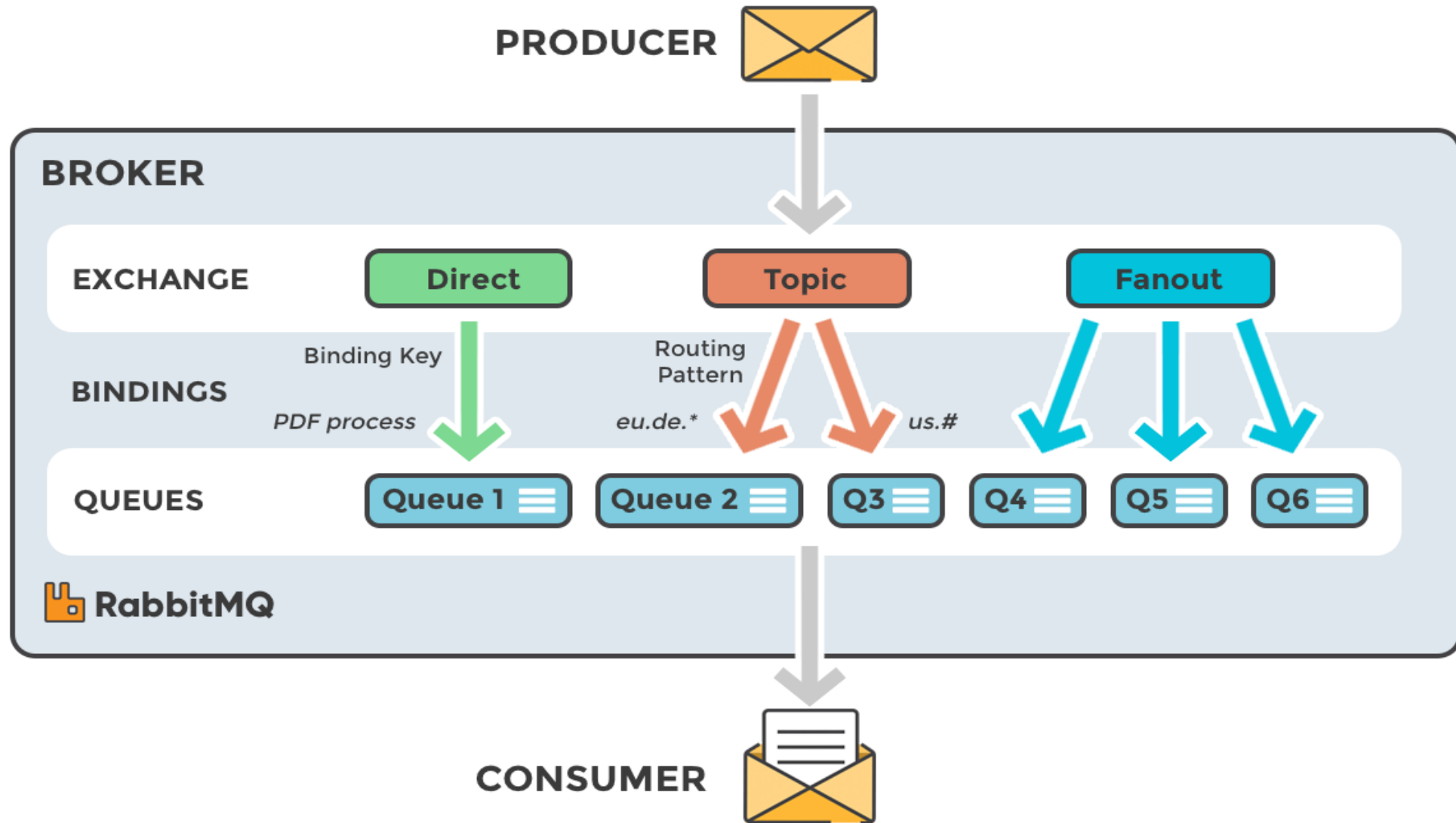
Communication between microservices - Asynchronous



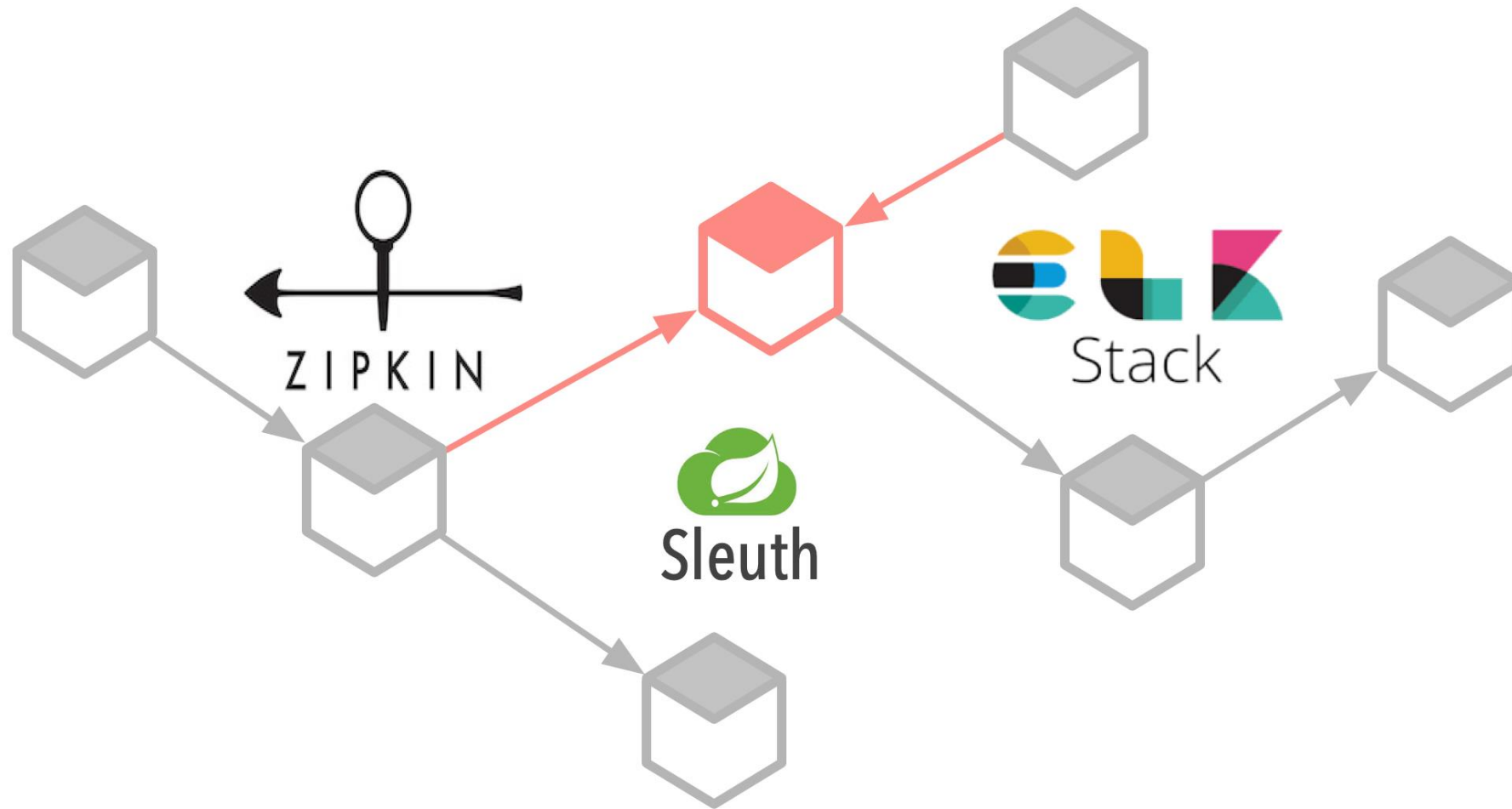
Communication between microservices - Asynchronous



Communication between microservices - Asynchronous



Distributed Tracing



What is Distributed Tracking?

A pattern to address debugging challenges in microservices architecture where a simple action can trigger a chain of microservices calls across different containers and/or environments

Tools Used



Library that tracks the progress by adding trace and span ids on HTTP request headers using Mapped Diagnostic Concept



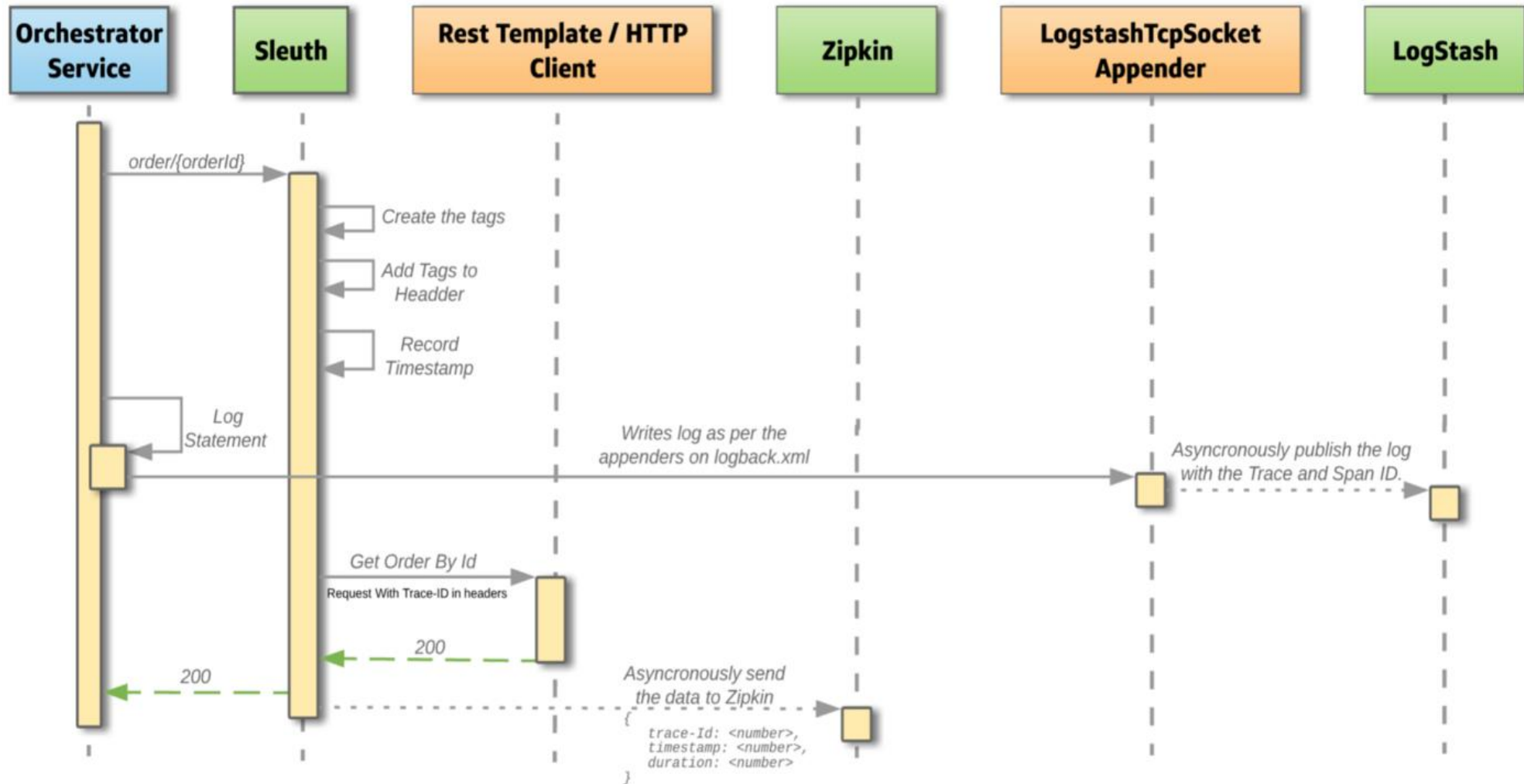
Java based application helps gather timing data of every request propagated between independent services. It has a management console for visualization of this timing statistics data



Set of tools for searching, analyzing and visualizing log data in real time

- i. Elasticsearch – Search and analytics engine
- ii. Logstash – Server side data processing pipeline that ingests data from multiple sources simultaneously, transforms it and sends it to stash
- iii. Kibana – Visualization of data in form of charts and graphs

How it works



What is Circuit Breaker?

A pattern which gives control over latency and failure between distributed services. It is used to stop cascading failures by failing fast and recover as soon as possible.

Why to use?

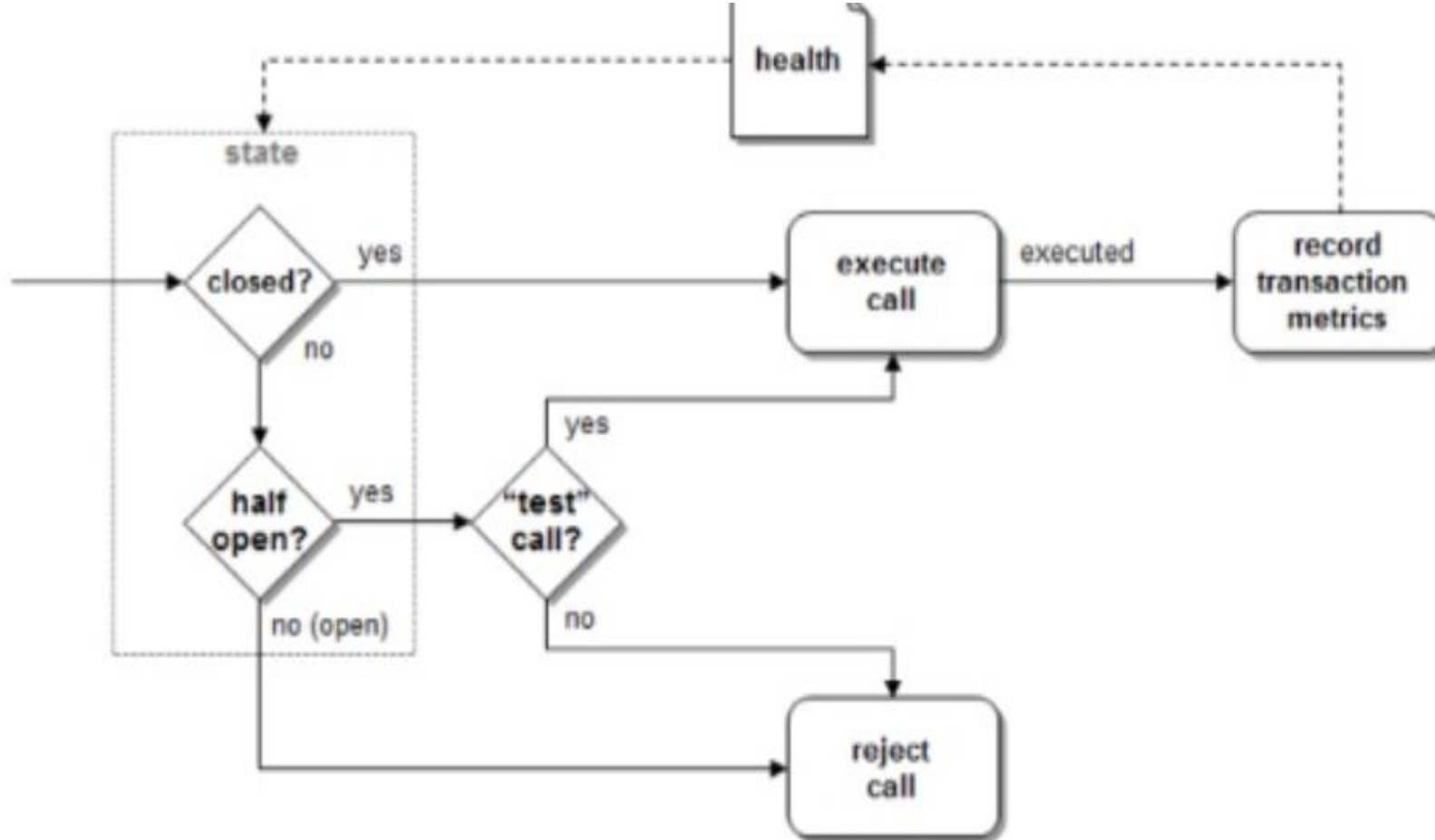
When one service synchronously invokes another there is always a possibility that the other service is unavailable or is exhibiting such latency that is essentially unusable. Previous resources such as threads might be consumed in the caller while waiting for other service to respond. This might lead to resource exhaustion leading to make calling service unable to handle other requests. The failure of one service can potentially cascade to other services throughout the application.

Tools Used



Hystrix is an open source framework developed by the company Netflix, facilitating latency and fault tolerance for distributed systems. At its core it provides an implementation of the circuit breaker pattern that you can easily utilize in your applications

How it works

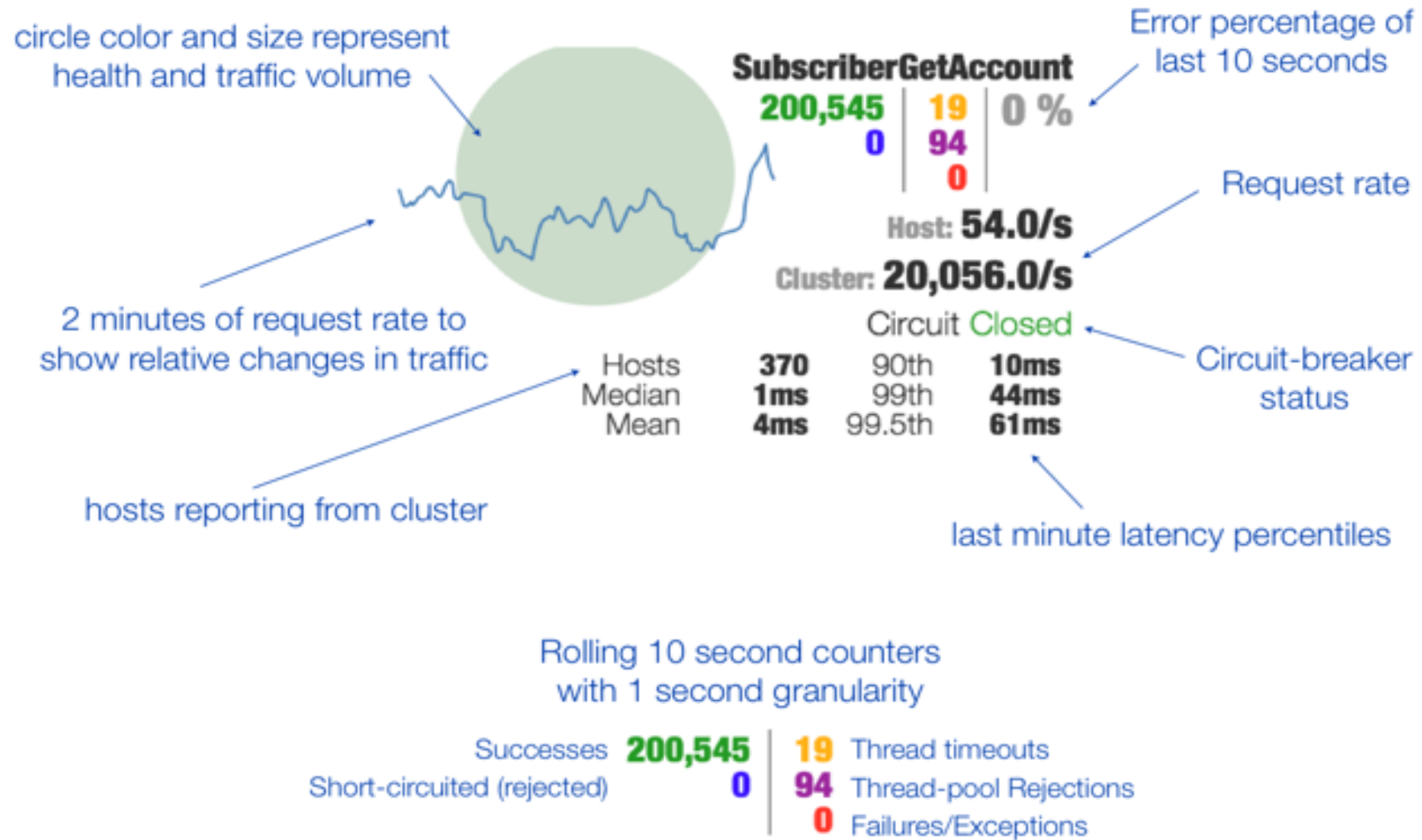


How it works

Circuit breaker can be in one of 3 states:

- **CLOSED** - When everything is fine, we just forward calls, as there would be nothing between the caller and wrapped code.
- **OPEN** - Each failure is recorded and, when a certain threshold is met, we disable call forwarding and start to fail immediately.
- **HALF OPEN** - While throwing errors, without even hitting backend, we do a check if grace period has passed. If it did, we try again. This time only once (now we're in this "HALF OPEN" state). If everything went well we go back to normal ("CLOSED") state or to failure ("OPEN") if failed. The main difference is that we don't check for the threshold. The first failure in "HALF OPEN" makes all go down again. No retries.

Hystrix Dashboard



What is CQRS?

A pattern that separates read and update operations for a data store.

Why to use?

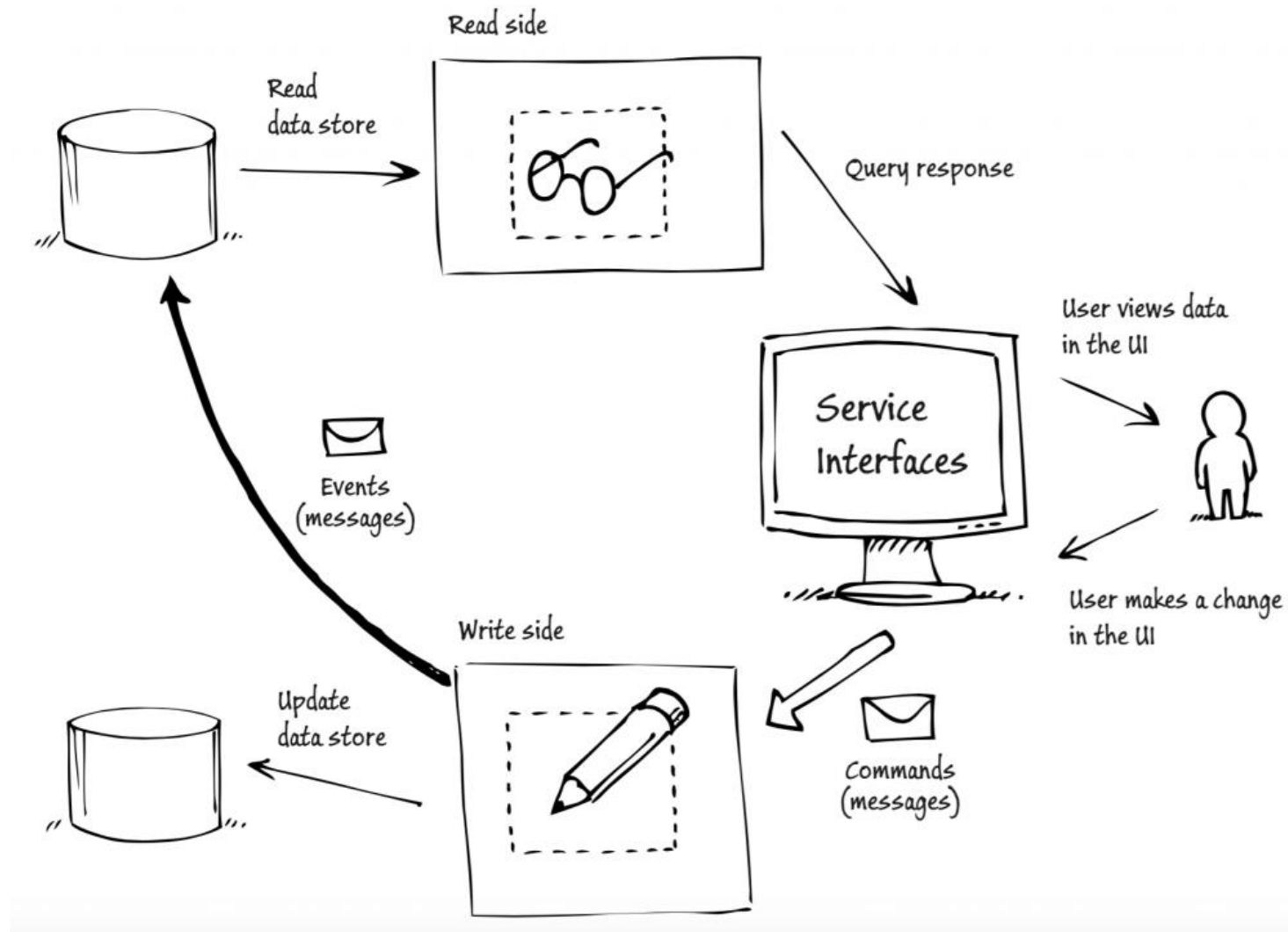
Independent scaling. CQRS allows the read and write workloads to scale independently and may result in fewer lock contentions.

Optimized data schemas. The read side can use a schema that is optimized for queries, while the write side uses a schema that is optimized for updates.

Security. It's easier to ensure that only the right domain entities are performing writes on the data.

Separation of concerns. Segregating the read and write sides can result in models that are more maintainable and flexible. Most of the complex business logic goes into the write model. The read model can be relatively simple.

How it works



Web Frameworks

python node.js

spring Java RAILS

django Flask

Frontend UI

ANGULARJS jQuery

Bootstrap HTML5

React

DevOps & Infra

puppet labs CHEF

ANSIBLE Jenkins

Spinnaker TERRAFORM

Database & Machine Learning

MySQL PostgreSQL CONSUL

mongoDB Cassandra redis

elastic TensorFlow

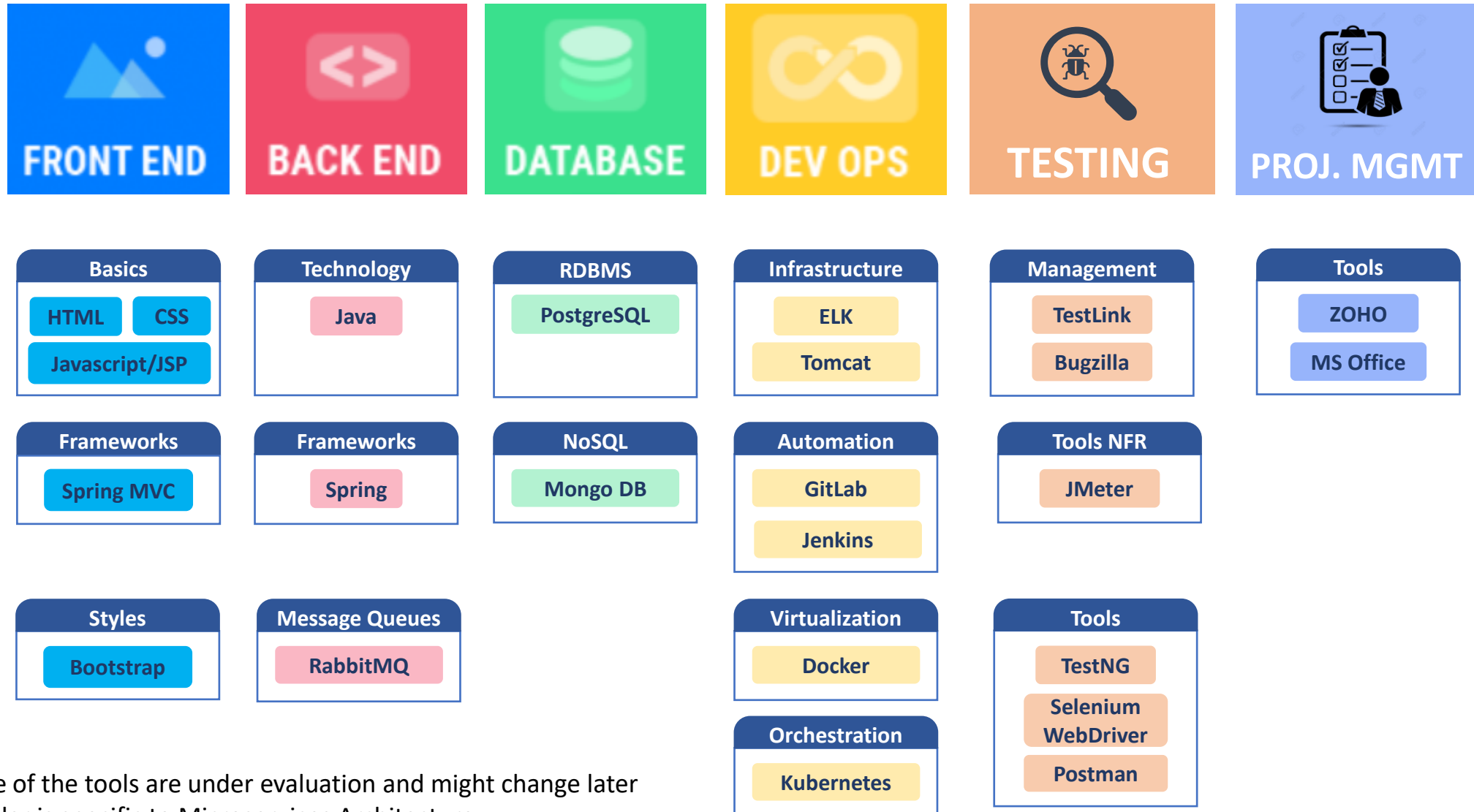
kafka hadoop Spark amazon REDSHIFT

Chat & Voice Bots

Cortana Amazon Lex twilio
skype amazon alexa twitter

Technology Stack

Stack – Technology and Tools



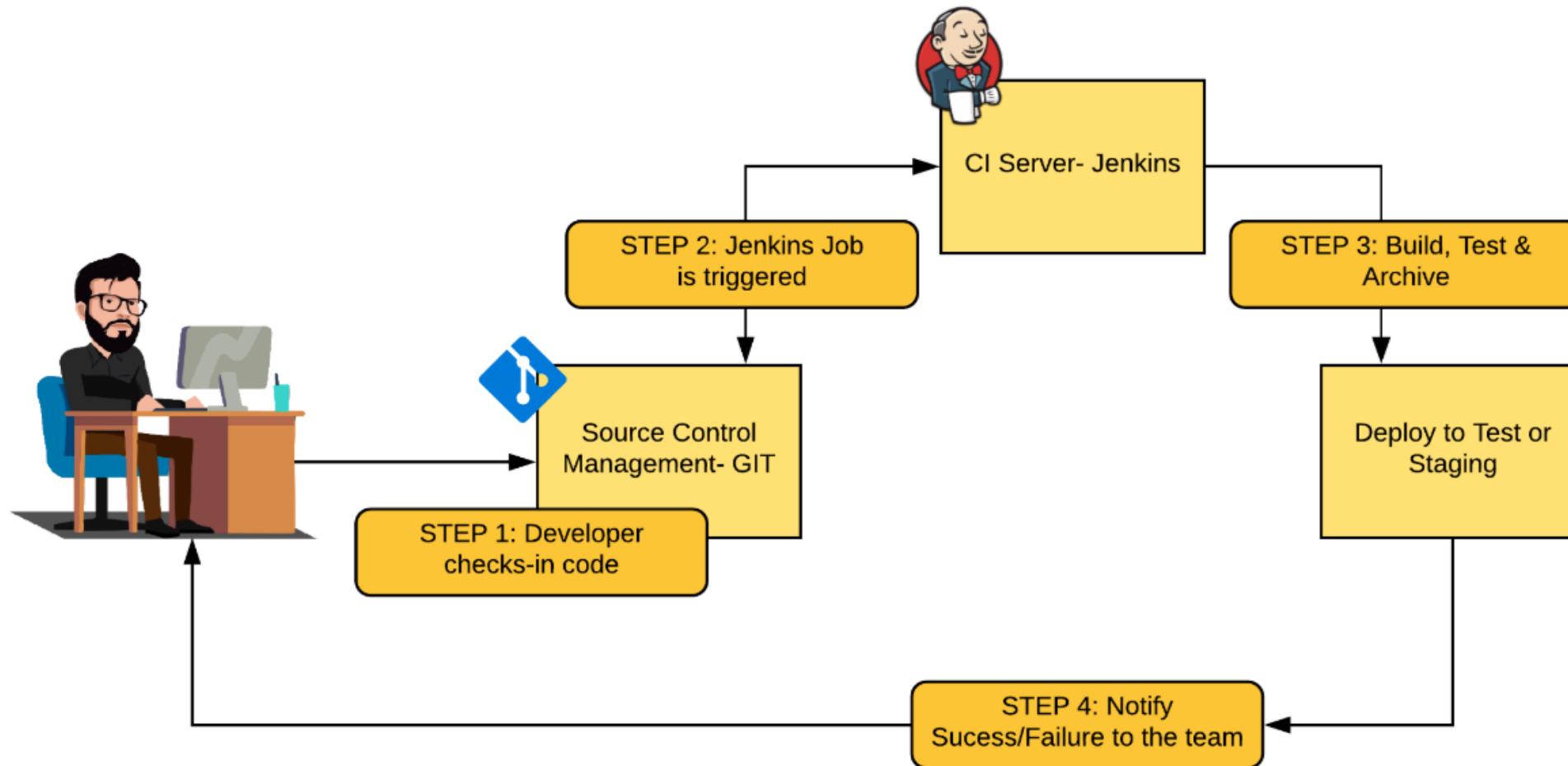
* Some of the tools are under evaluation and might change later

* Angular is specific to Microservices Architecture



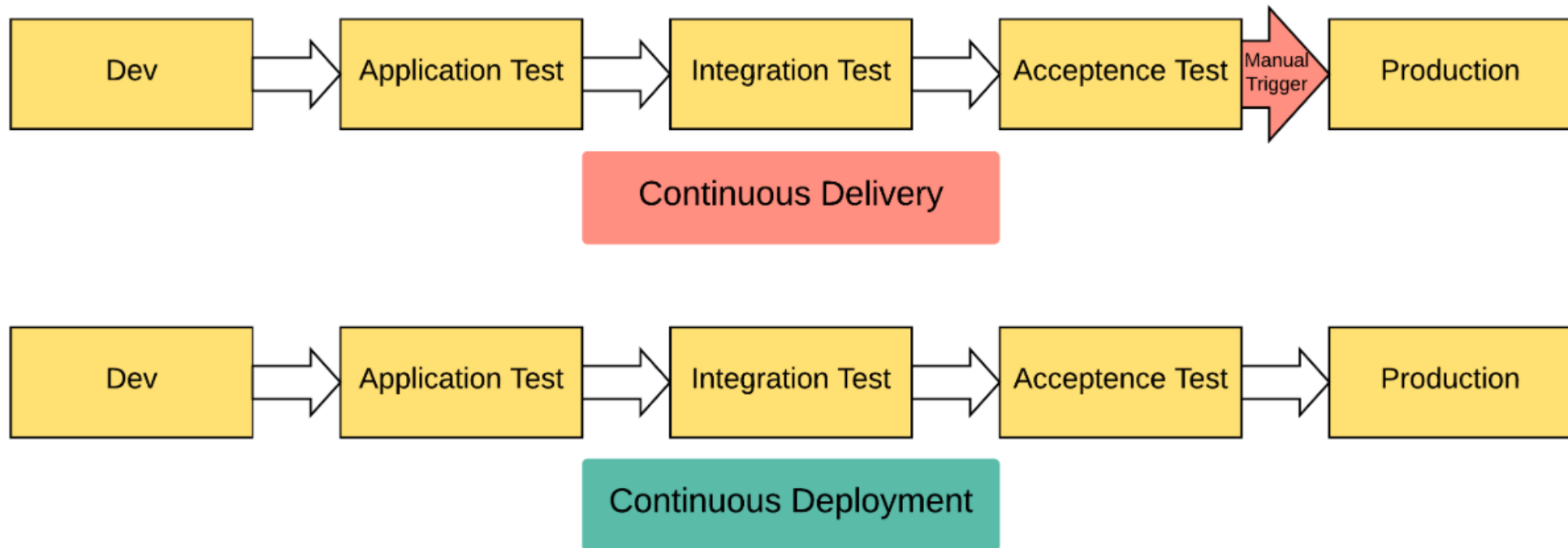
DevOps - CI

Continuous Integration

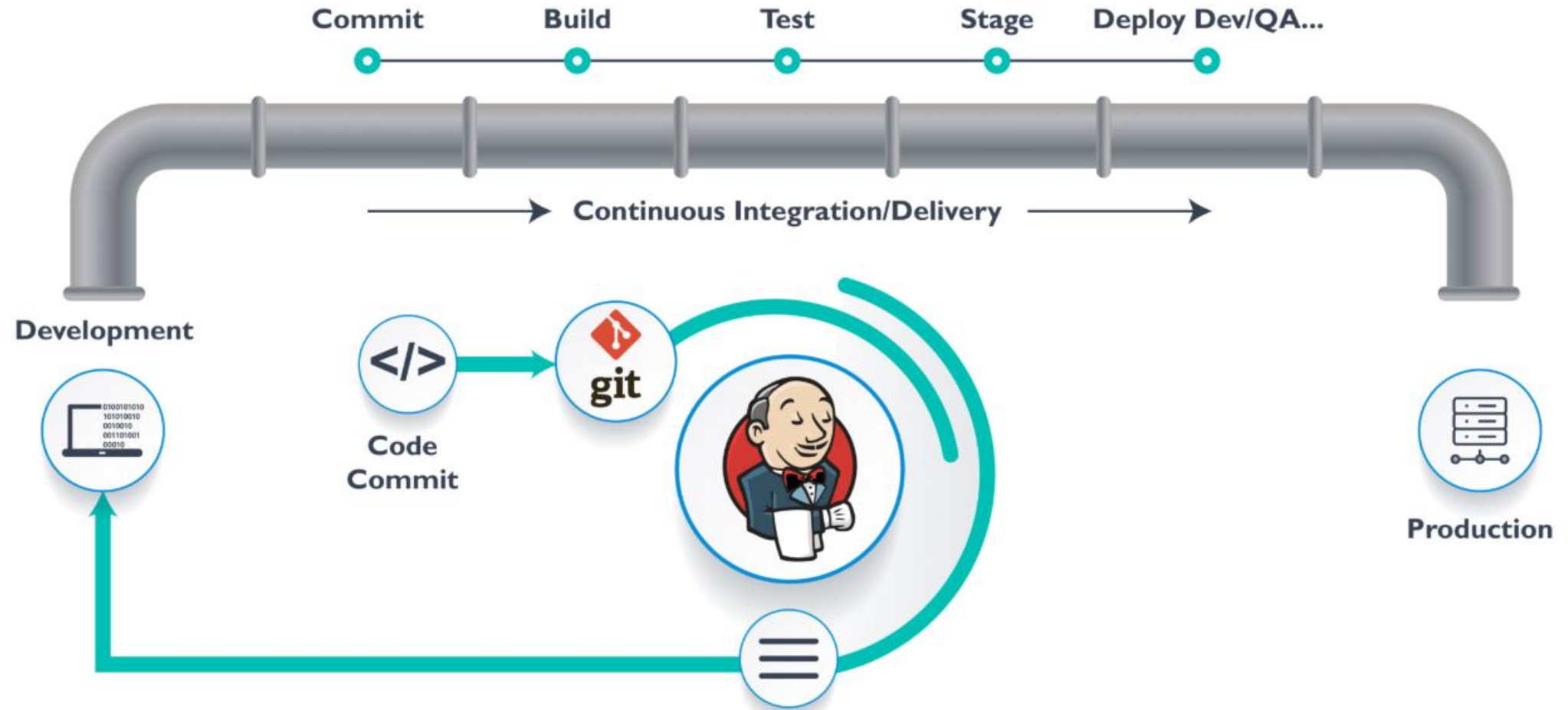


DevOps -CD

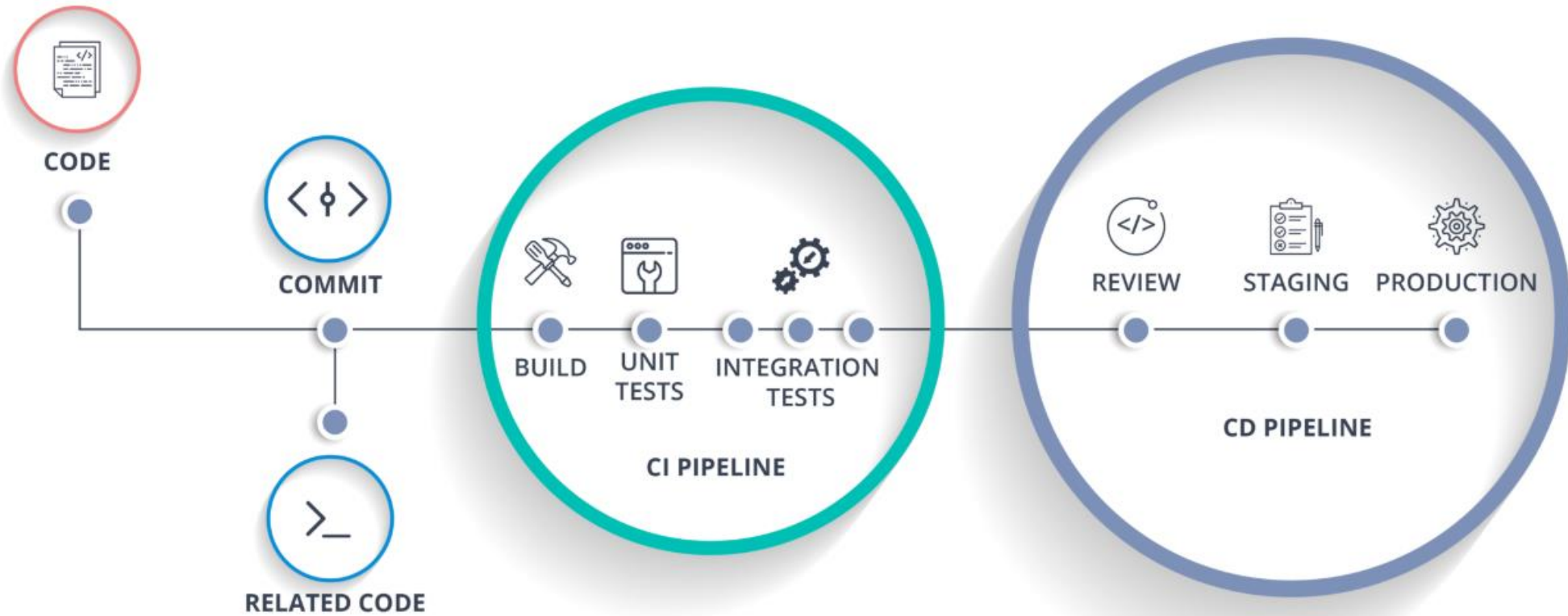
Continuous Delivery vs Continuous Deployment



Pipeline stages

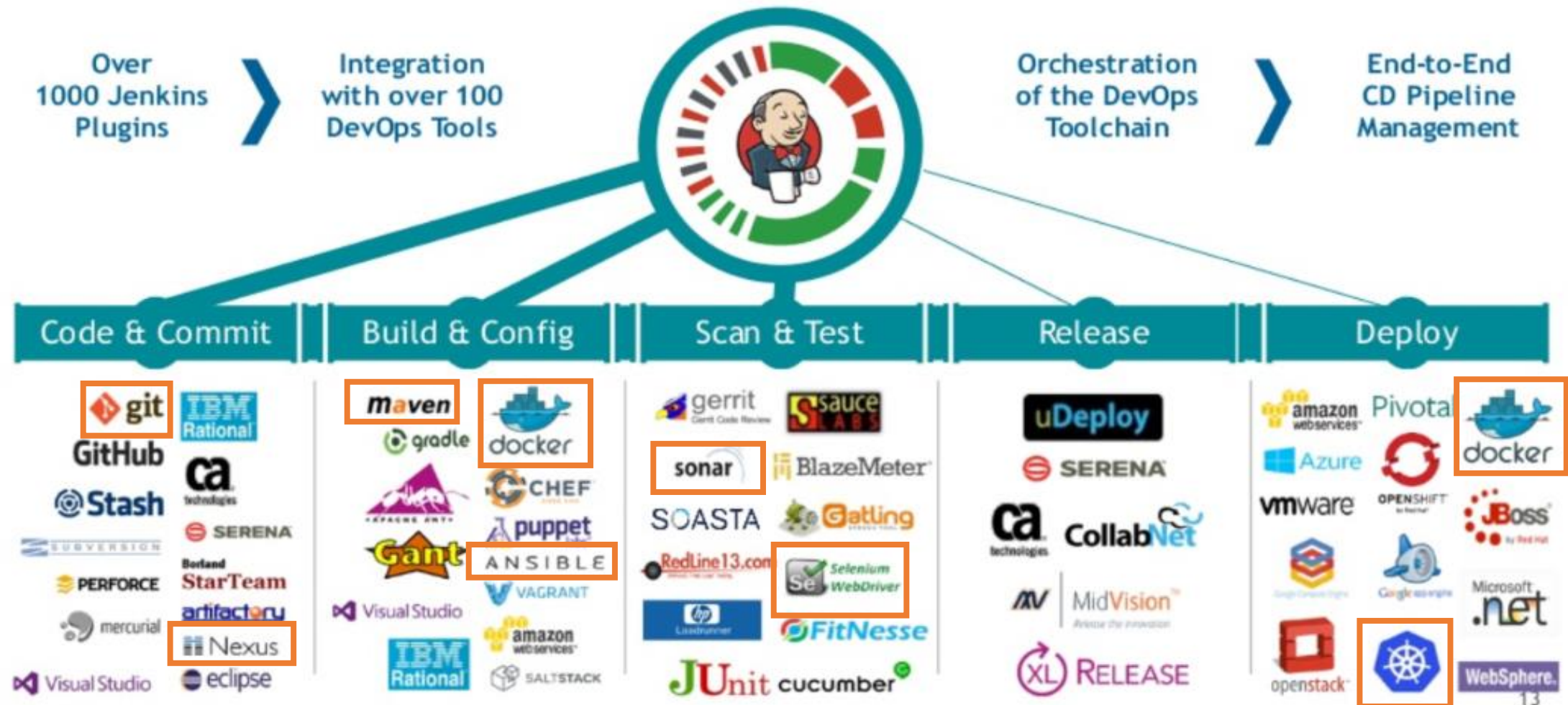


CI-CD

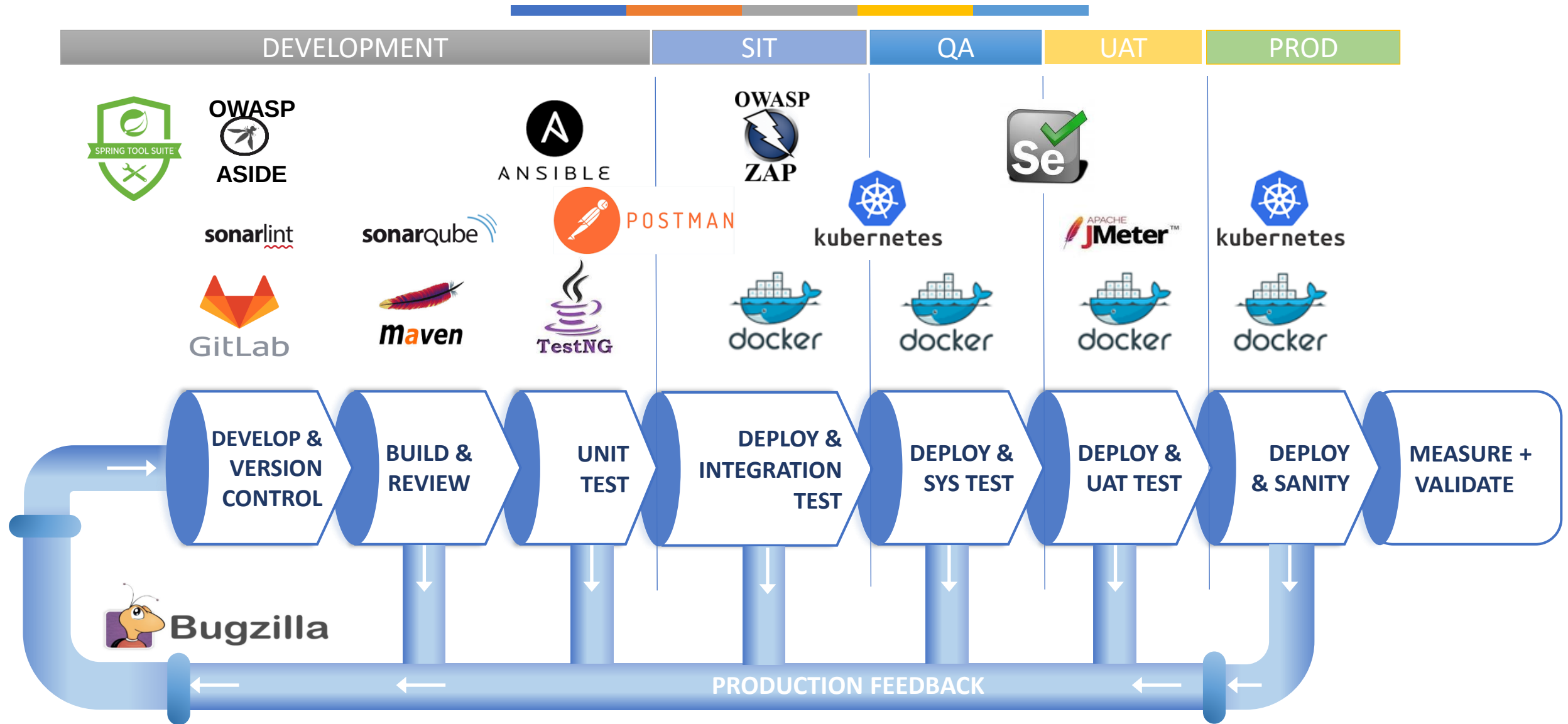


CI-CD Tools

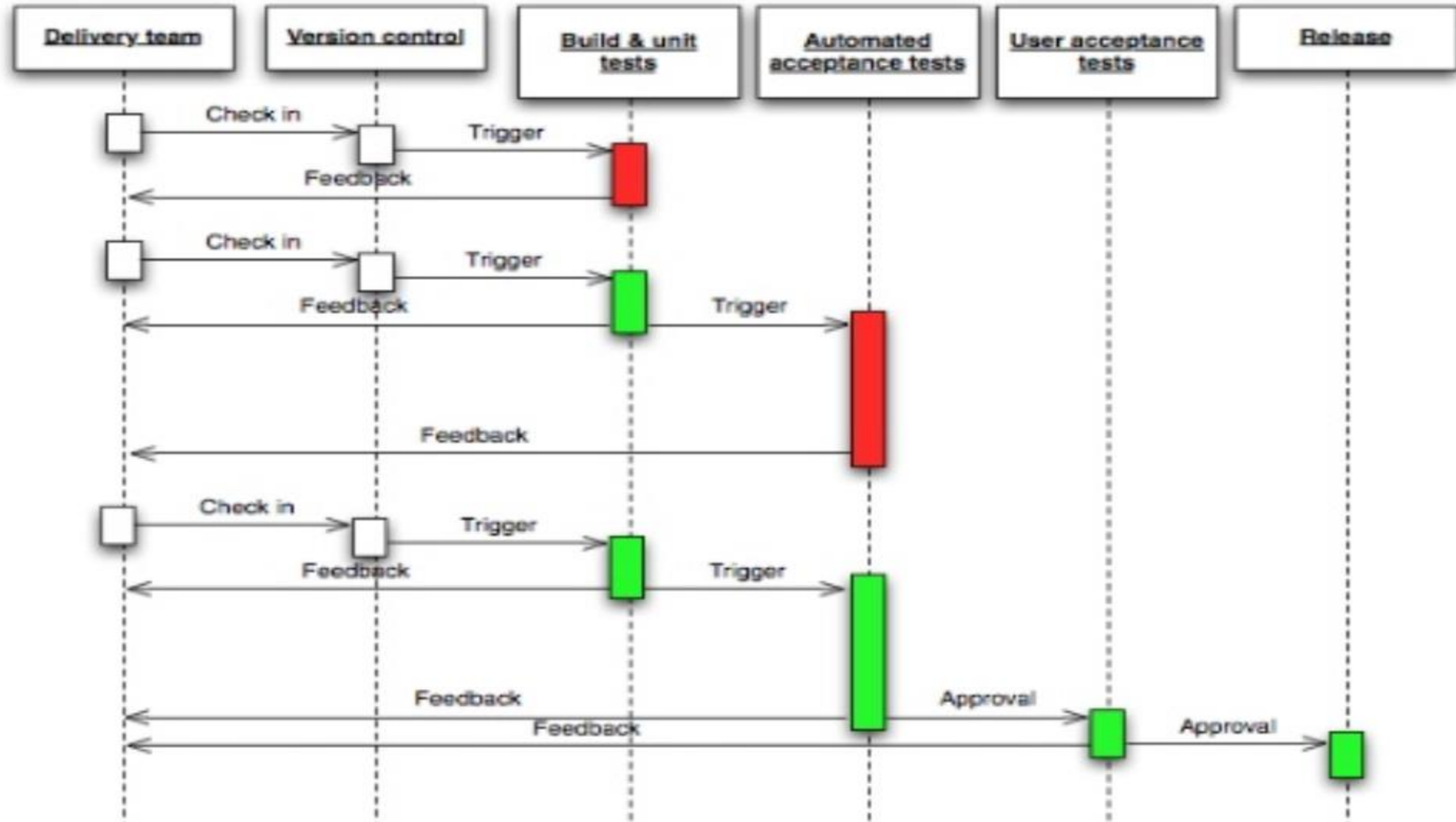
Jenkins is the Hub of the CD/DevOps Ecosystem



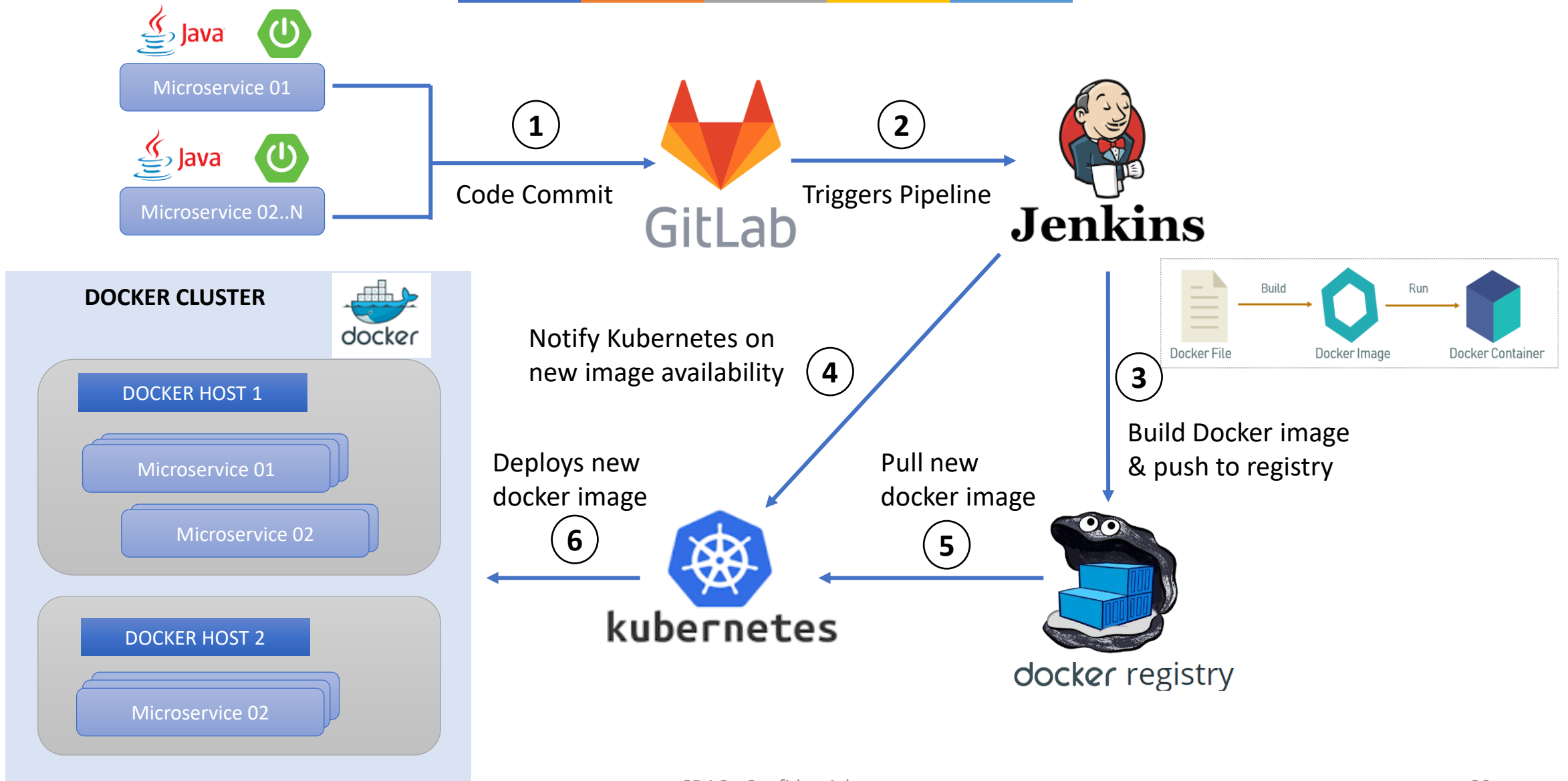
CI-CD SMILE



CI-CD Process Flow



Containerization



Development process

Environments



- Servers
 - Development (DEV)
 - System Integration Testing (SIT)
 - Testing (QA)
 - User Acceptance Testing (UAT)
 - Production (PROD)
- Clients
 - Individual desktops

Software



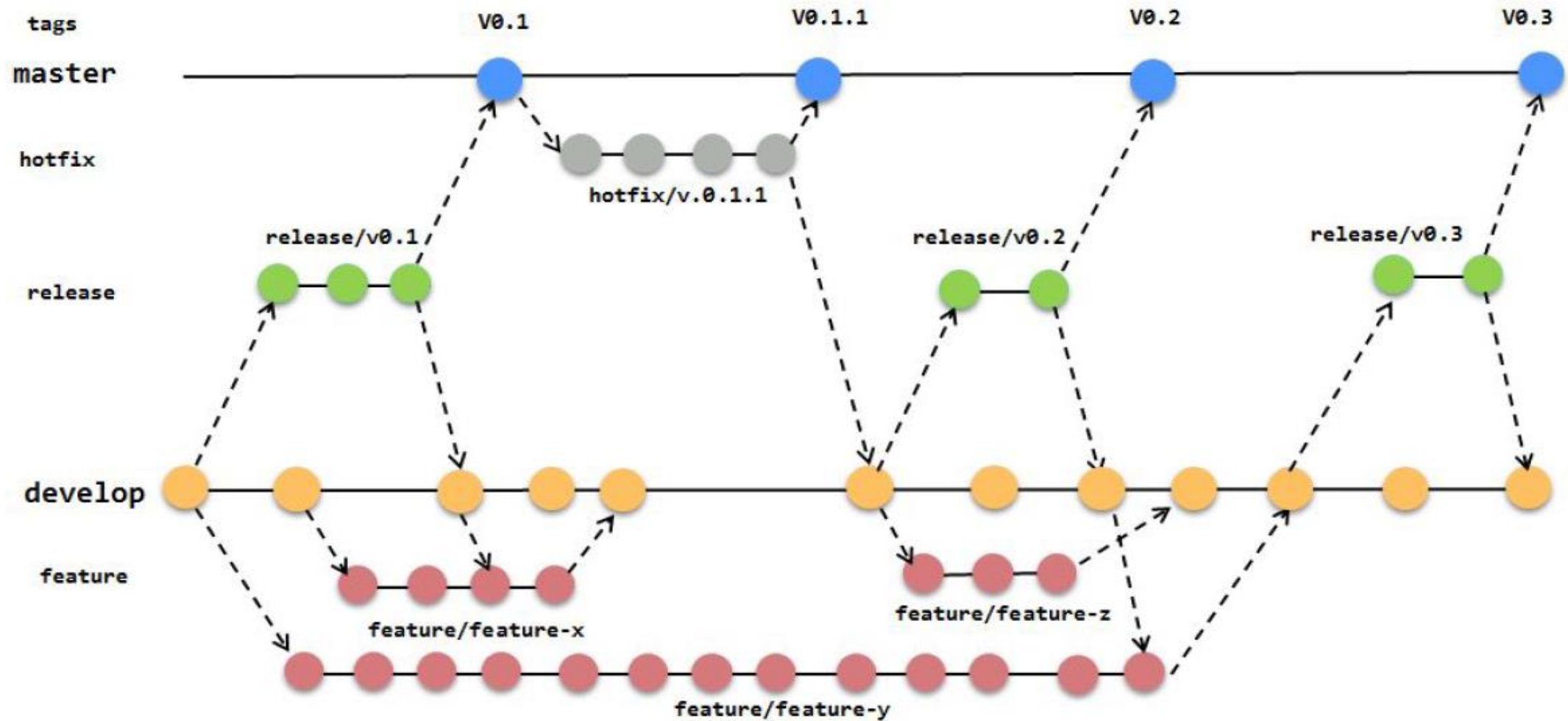
- Software versions provided as per Tool set
- Pushed into Developer's client machines centrally by ITSS
- Upgrades to the software versions will also be pushed similarly

Version Control



- Version control will be done by GitLab
- Each microservice will have its own repository
- Only developers working on a microservice will be given r-w access
- All other developers will get r-o access to other repositories
- Will use GitFlow for branching

Branching



Branch Descriptions



MASTER

The master branch at origin should be familiar to every Git user. Parallel to the master branch, another branch exists called develop.

DEV

Primary development branch, where both teams are primarily committing to.

RELEASE

Release branch supporting preparation of a new production release. It allows minor bug fixes and preparing meta-data for a release (version number, build dates, etc.). By doing all of this work on a release branch, the develop branch is cleared to receive features for the next big release.

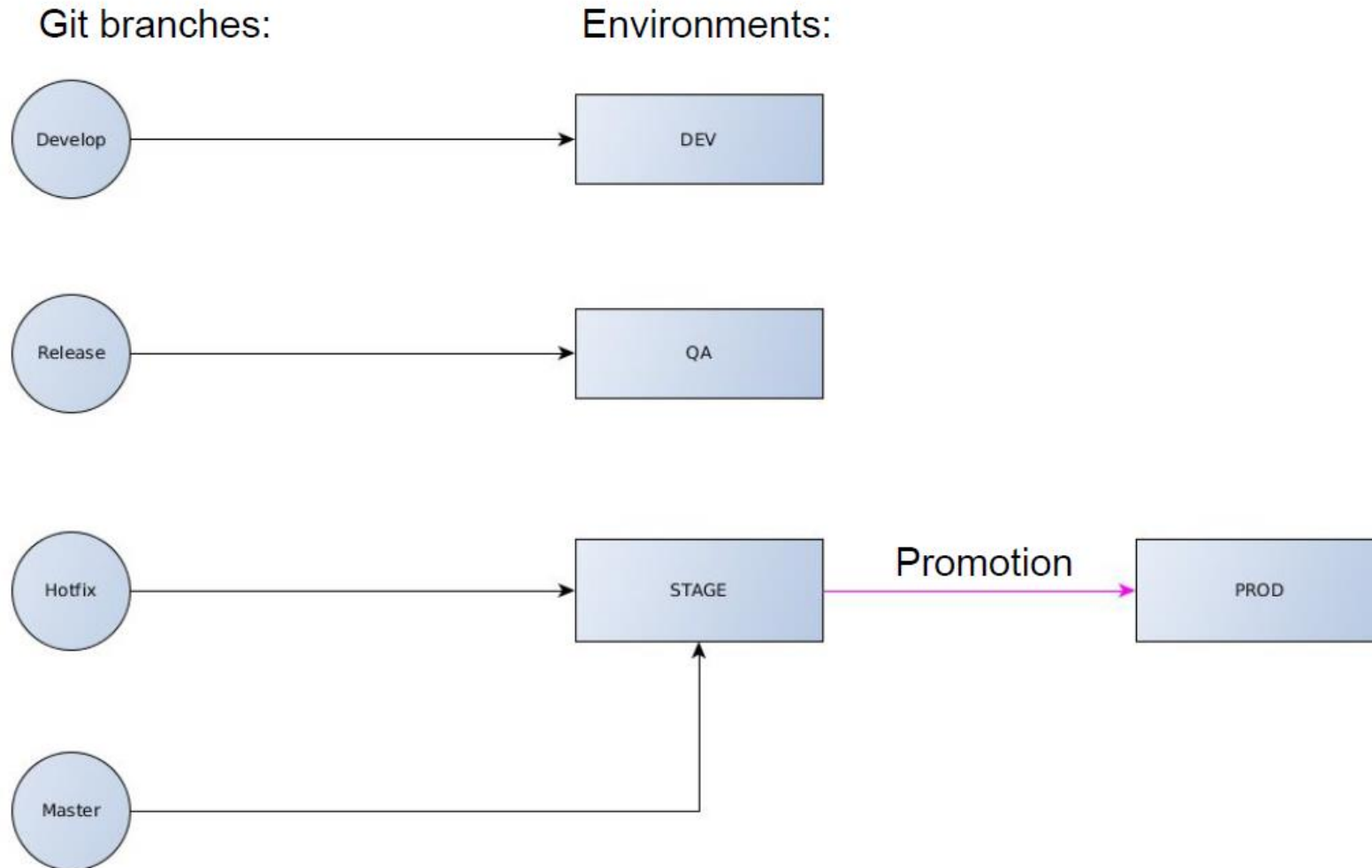
HOTFIX

Hotfix branch is very much like release branch in that it is also meant to prepare for a new production release, that is unplanned. It arises from the necessity to act immediately upon an undesired state of a live production version. When a critical bug in a production version must be resolved immediately, a hotfix branch may be branched off from the corresponding tag on the master branch that marks the production version.

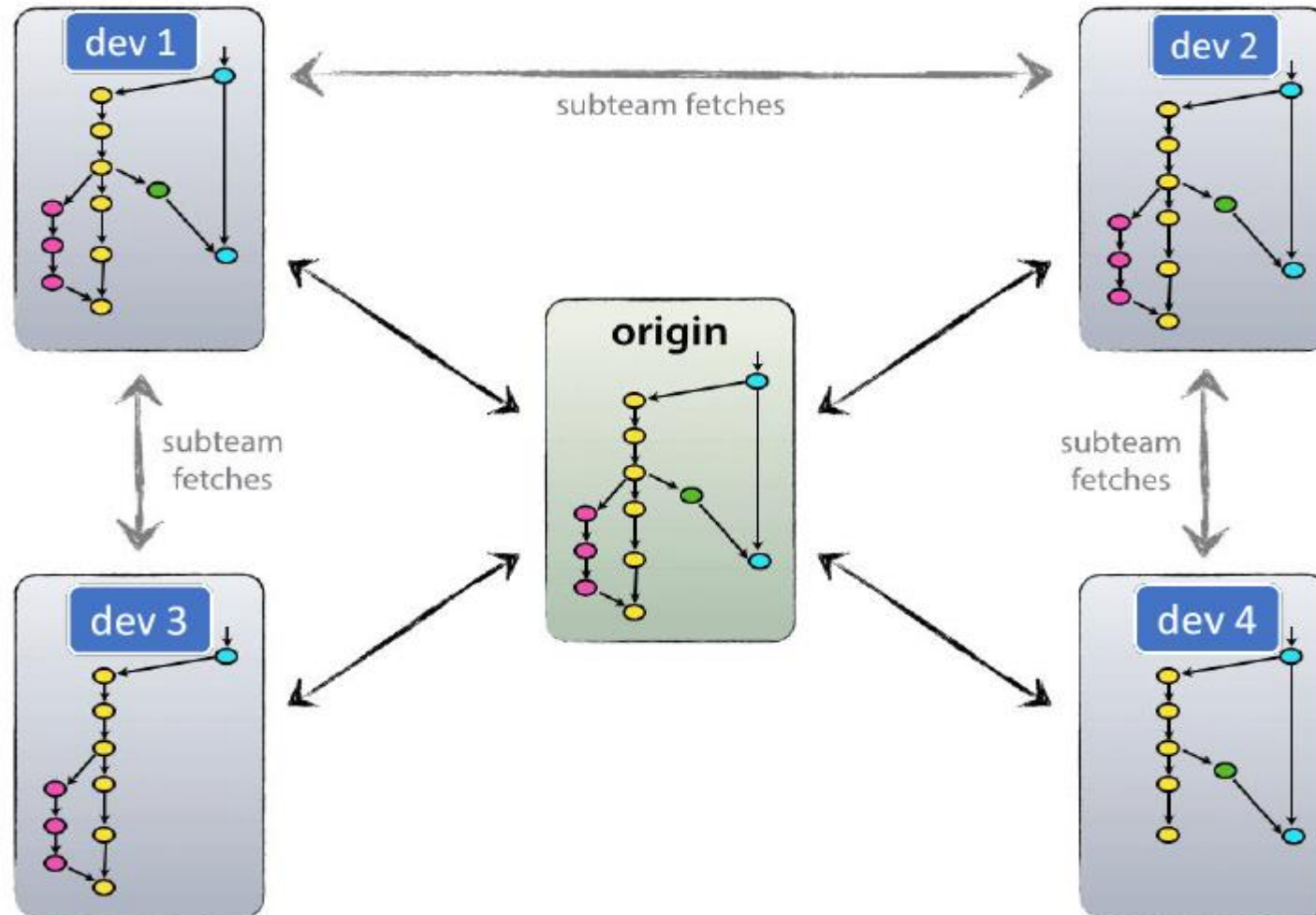
FEATURE

Feature branches (or sometimes called topic branches) are used to develop new features for the upcoming or a distant future release. When starting development of a feature, the target release in which this feature will be incorporated may well be unknown at that point. The essence of a feature branch is that it exists as long as the feature is in development but will eventually be merged back into develop.

Branches Mapping to Environments



Branches Mapping to Environments



- Each Developer pulls and pushes to origin
- But besides the centralized push-pull relationships, each developer may also pull changes from other peers to form sub-teams

Project Structure



- The project structure will be as created by STS Spring Boot application
- The folders will be created as per specified project structure
- The packages will be named as per specified naming convention

Code Quality

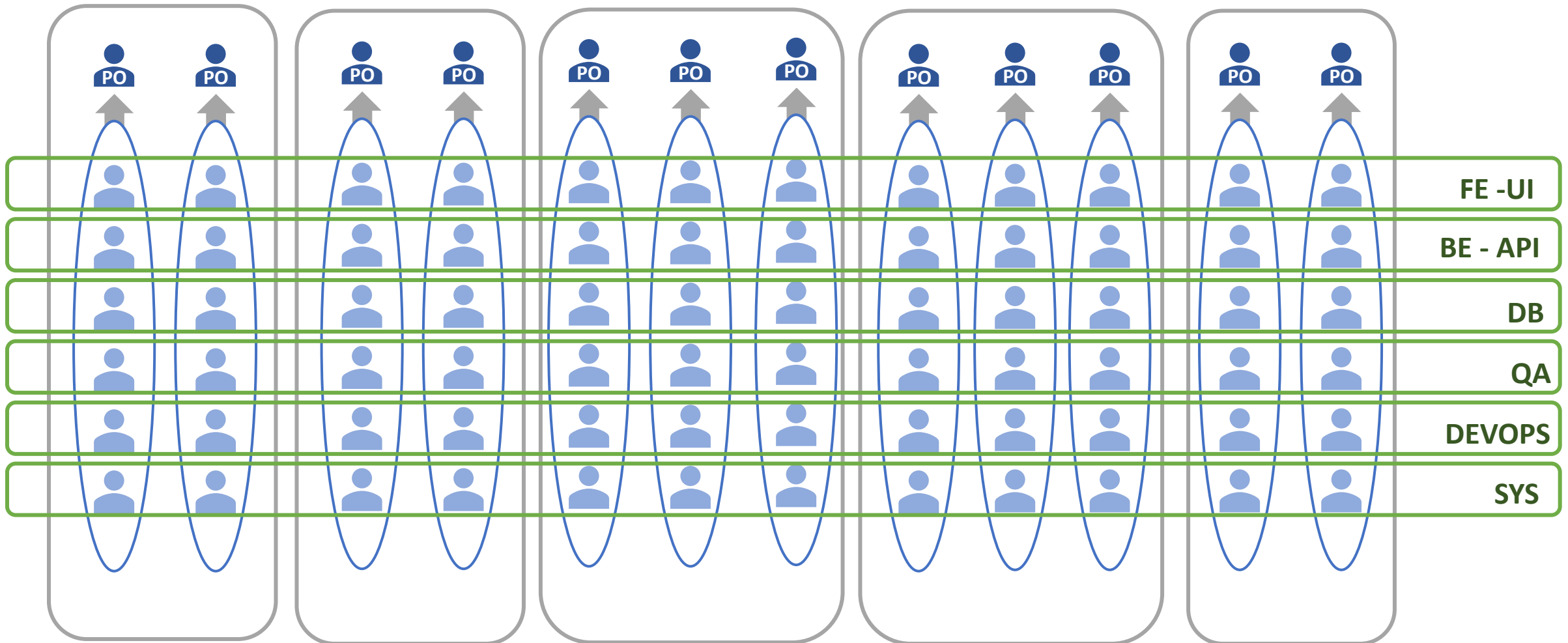


- Quality gates are defined on SonarQube to check for quality of Code.
- Developers run local Sonar tests before committing the code to VCS.
- CI tool is integrated with Sonar. Build jobs would not succeed if code quality does not meet the thresholds defined on Quality Gates.
- Offers reports on duplicated code, code quality, unit tests, code complexity, code coverage.
- Integrate with build tools: Maven, Gradle, etc.
- Integrate with different IDE for local Code quality tests.

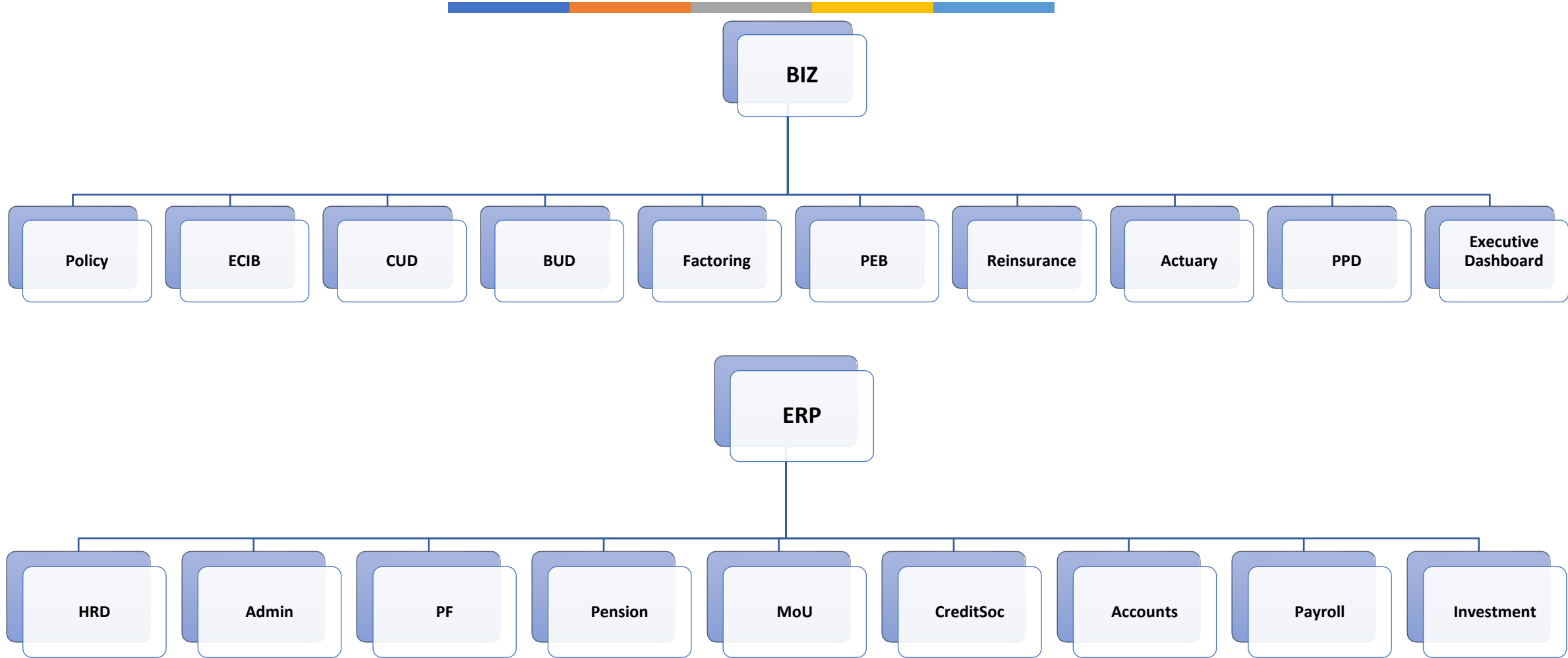


Team Structure

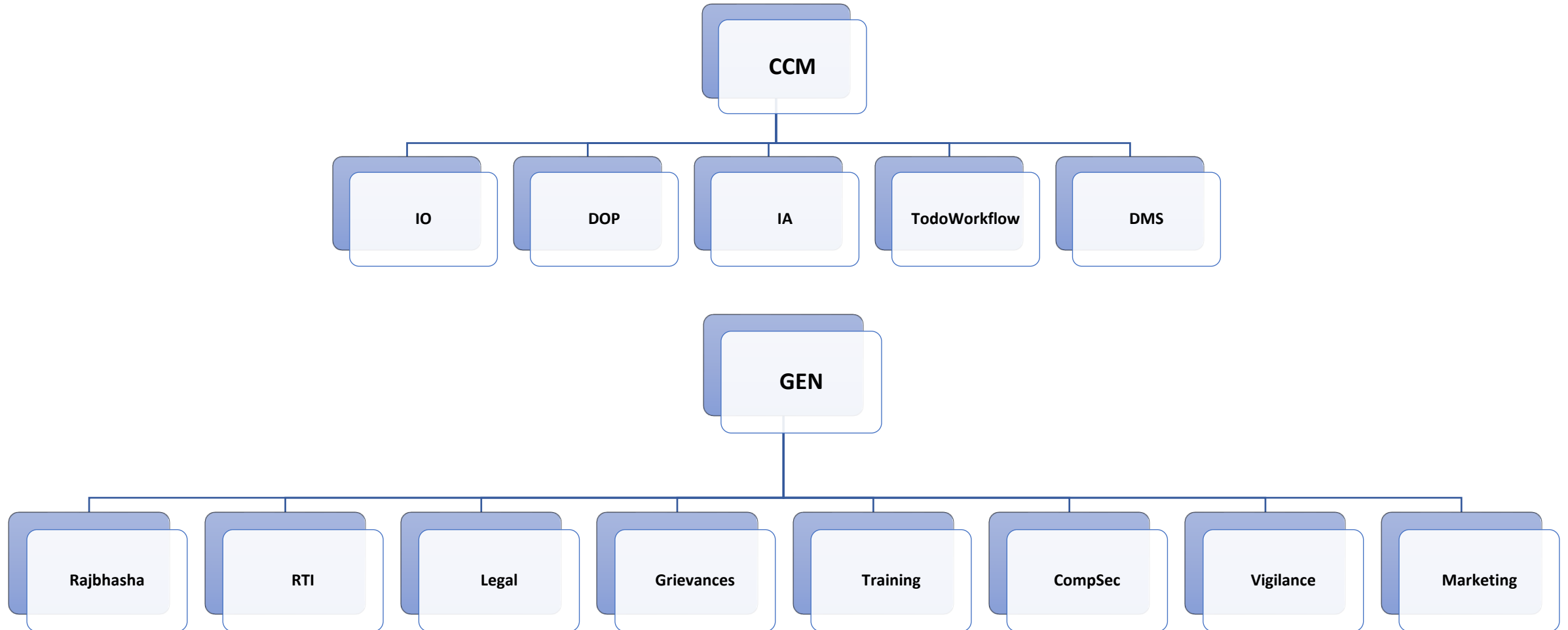
Team Structure



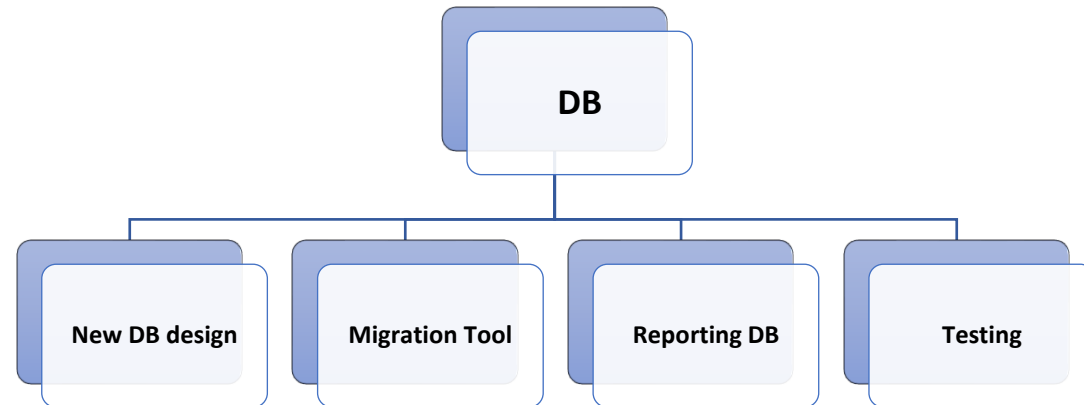
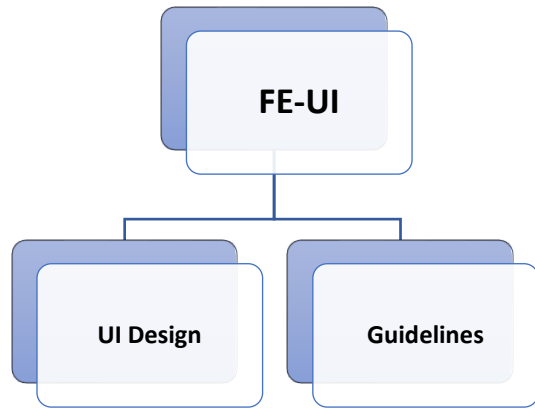
Team



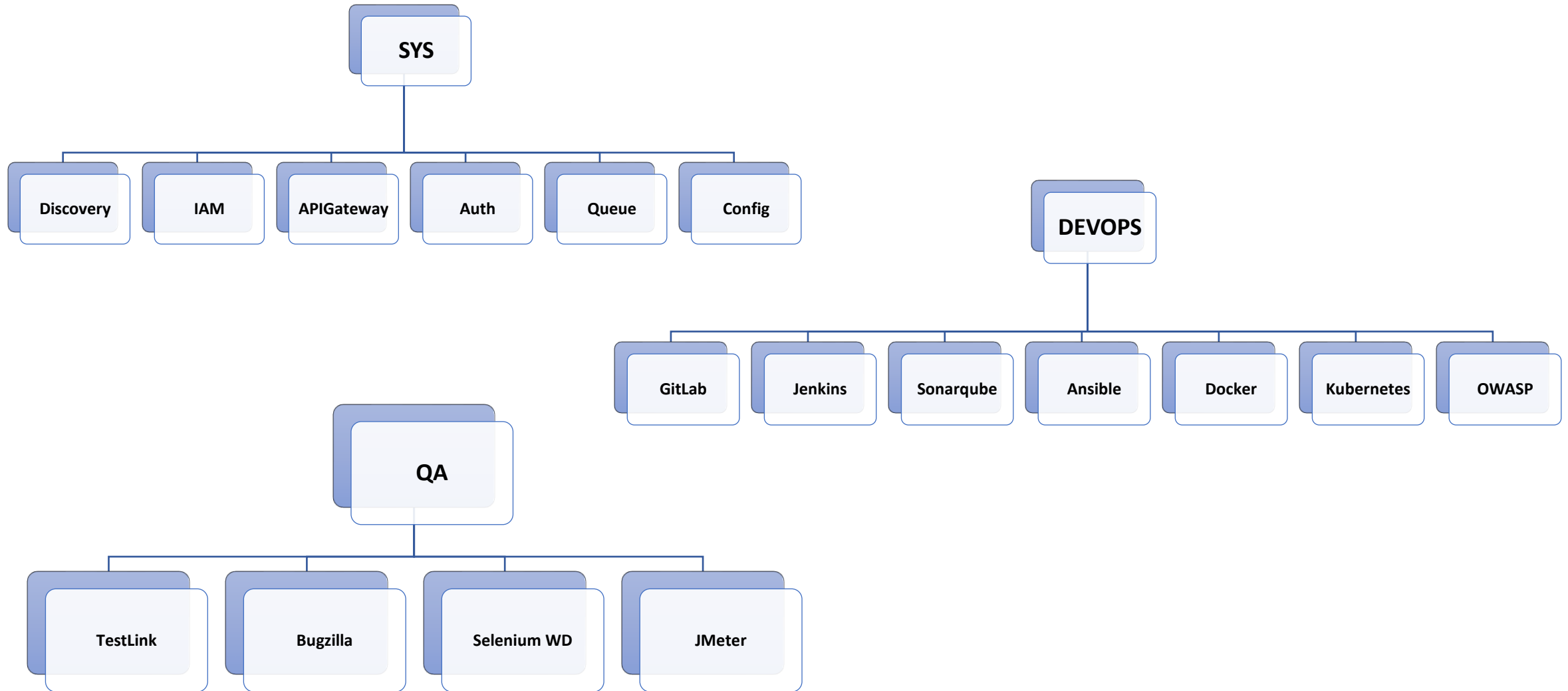
Team

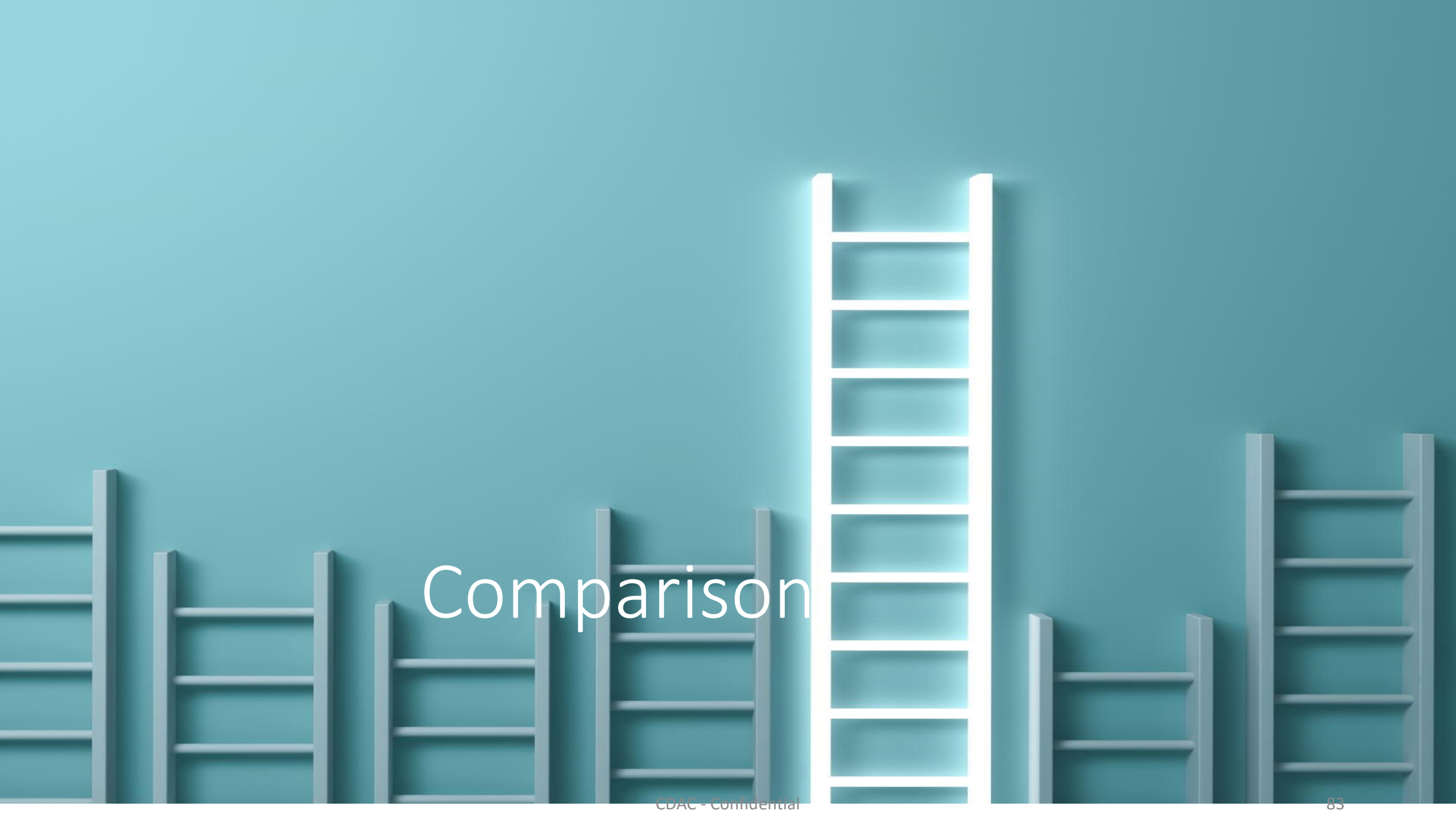


Team



Team





Comparison

Design – Monolithic vs Microservices

	MONOLITHIC	MICROSERVICES
1	Strongly coupled services with low cohesion	Designing high cohesive and de-coupled services is difficult
2	With single database data consistency is easier to maintain	Data consistency and availability while scaling above is complex with multiple databases
3	Scope of design autonomy limited to a single technology stack.	Higher design autonomy. Team has greater freedom to select technology appropriate for the business needs in general.
4	Strict design discipline necessary to avoid spaghetti code during development as well as maintenance	By design, the functionality is isolated within services and its database
5	Larger impact of re-designing if underlying technology is not supported	Impact restricted to the services where the technology is outdated
6	Additional effort required to make design secure	By very nature the design is secure

Development – Monolithic vs Microservices

MONOLITHIC		MICROSERVICES
1	Single Large Codebase	Multiple Codebases
2	Single Language and database	Polyglot and multiple databases
3	Code Complexity	Code isolated by module / services
4	Low Cohesion and Strong Coupling	High Cohesion and Loose Coupling
5	Feature Delays	Quicker feature implementation
6	Longer Development times with increase in functionality of the system	Shorter development times
7	Slow IDE load	Faster Individual project load with flexibility for tool

Deployment – Monolithic vs Microservices

MONOLITHIC		MICROSERVICES
1	Single Deployment (tightly coupled)	Services deployed independently (loosely coupled)
2	Easy (single JAR/WAR/EAR file)	Complex (lot of services with individual databases)
3	Static (Offline)	Dynamic (Live)
4	Slow	Fast
5	Downtime for entire application	No full application downtime Downtime only for impacted service(s)
6	Single point of failure	Only problematic service(s) affected
7	Rollback is time consuming	Rollback is quick

Maintenance – Monolithic vs Microservices

	MONOLITHIC	MICROSERVICES
1	Minor change could result in complete rebuild	Only impacted service would be rebuilt and deployed
2	Scaling requires duplication of the whole	Only required service can be scaled
3	Long build/test/release cycles	Continuous integration and deployment
4	Enables single/module team	Enables distributed teams
5	Decreased uptime with updates	Increased uptime
6	Regression testing can delay the updates	Testing focused on impacted services only
7	Whole system crash possible	Graceful degrade in case of issues

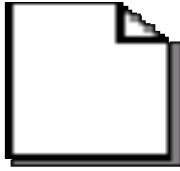
Infrastructure – Monolithic vs Microservices

MONOLITHIC		MICROSERVICES
1	On-Premise	Cloud (Private for ERP and Portal)
2	Less moving parts	Multiple moving parts

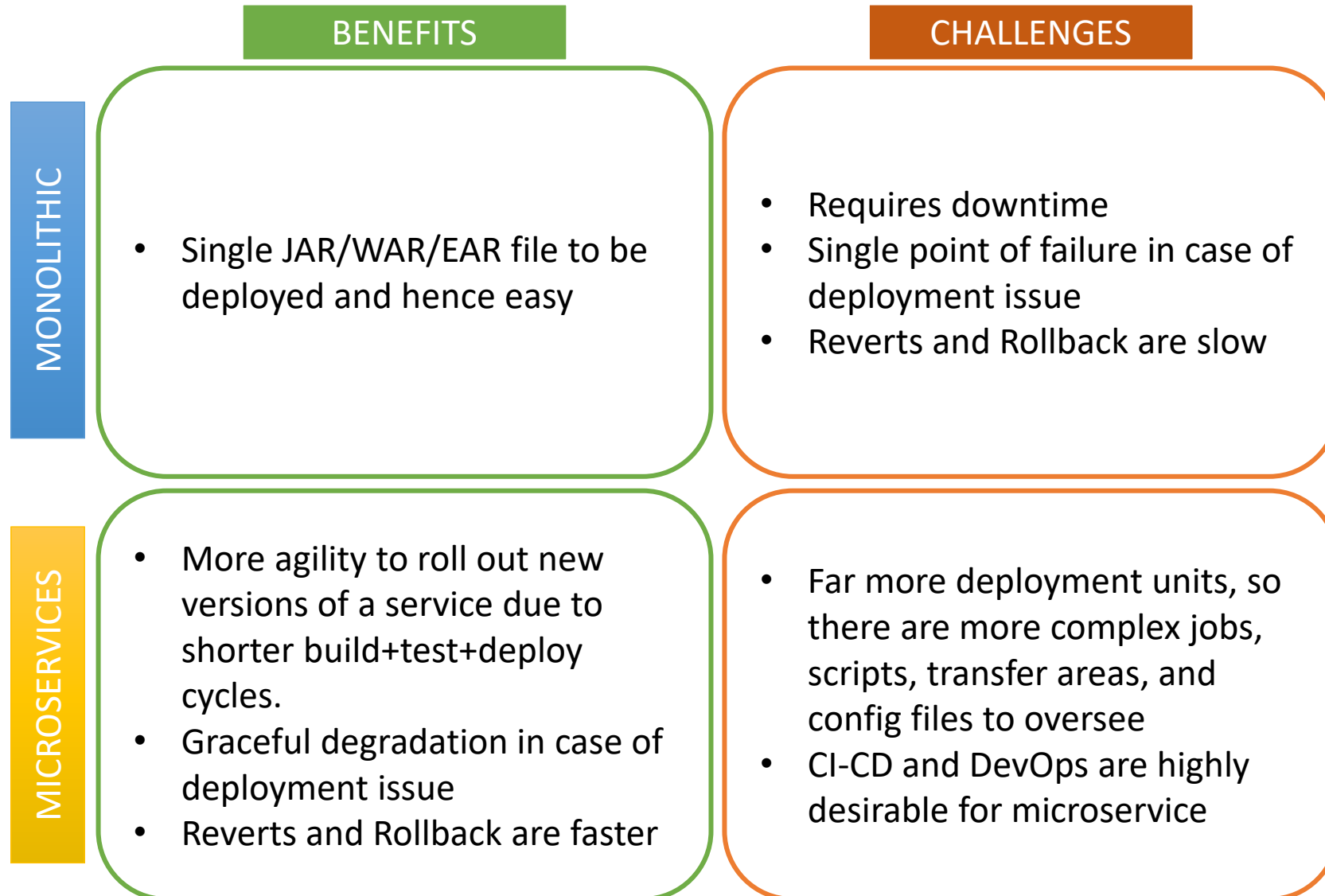


Evaluation & Recommendation

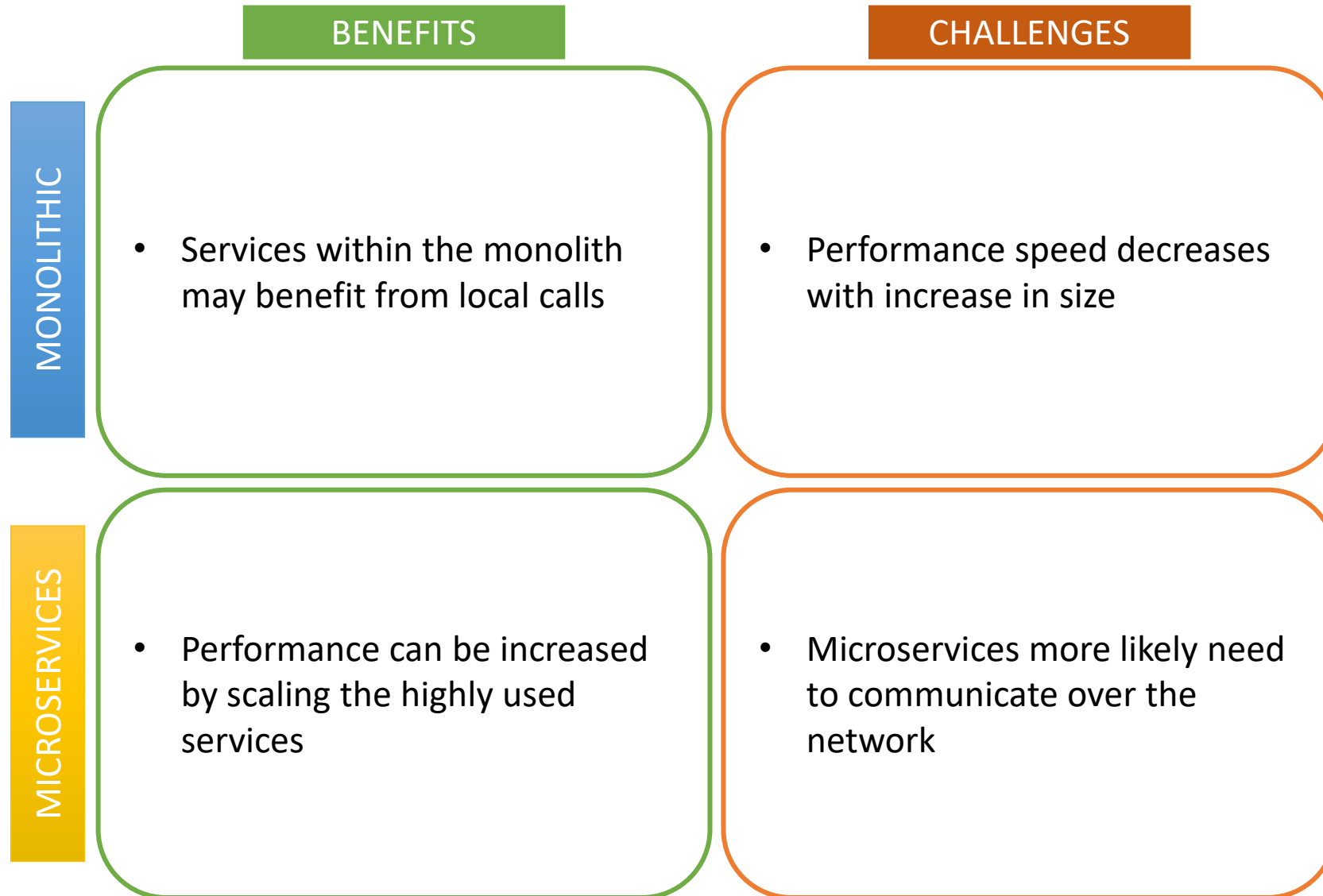
NFRs – Questionnaire and responses



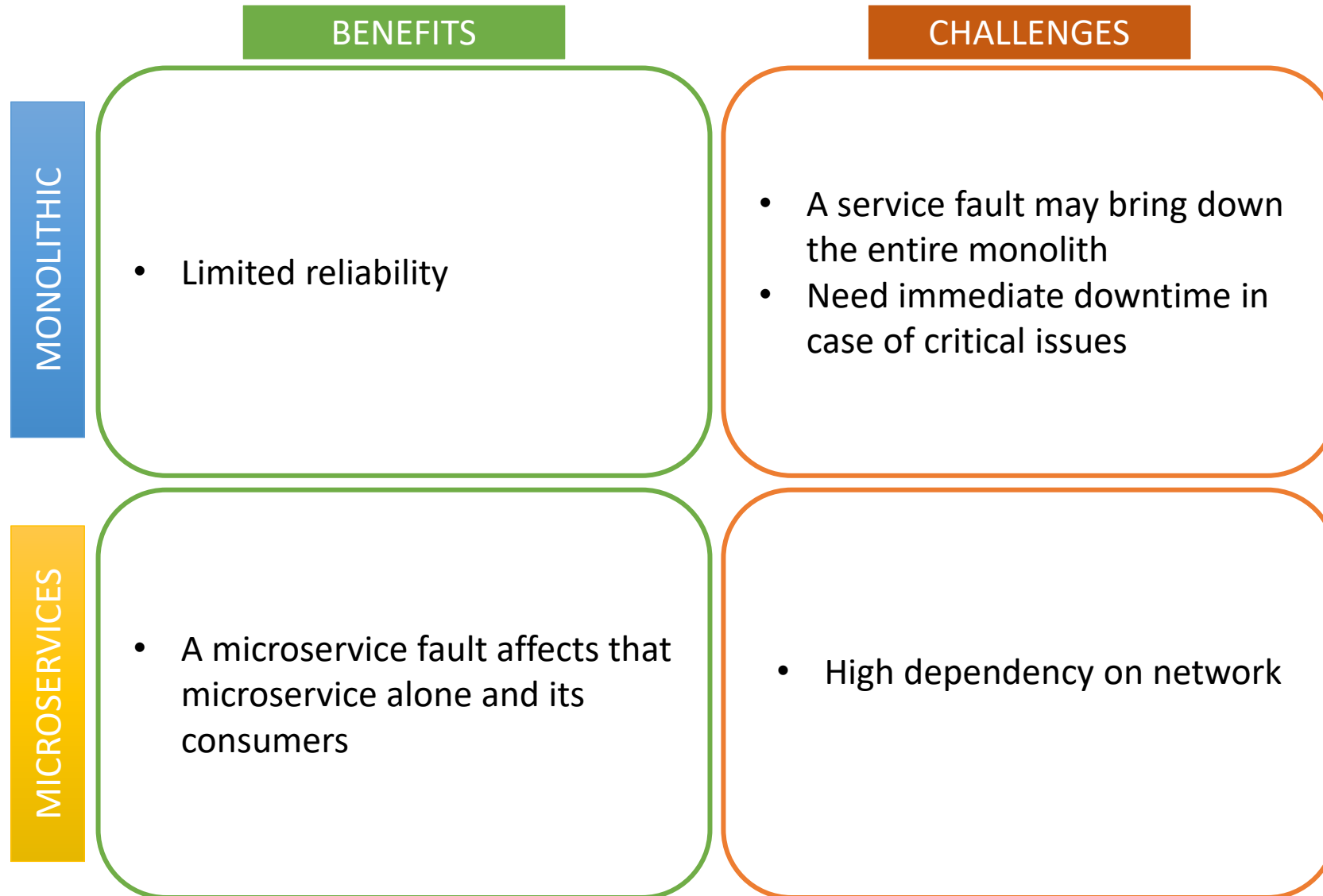
Deploy-ability



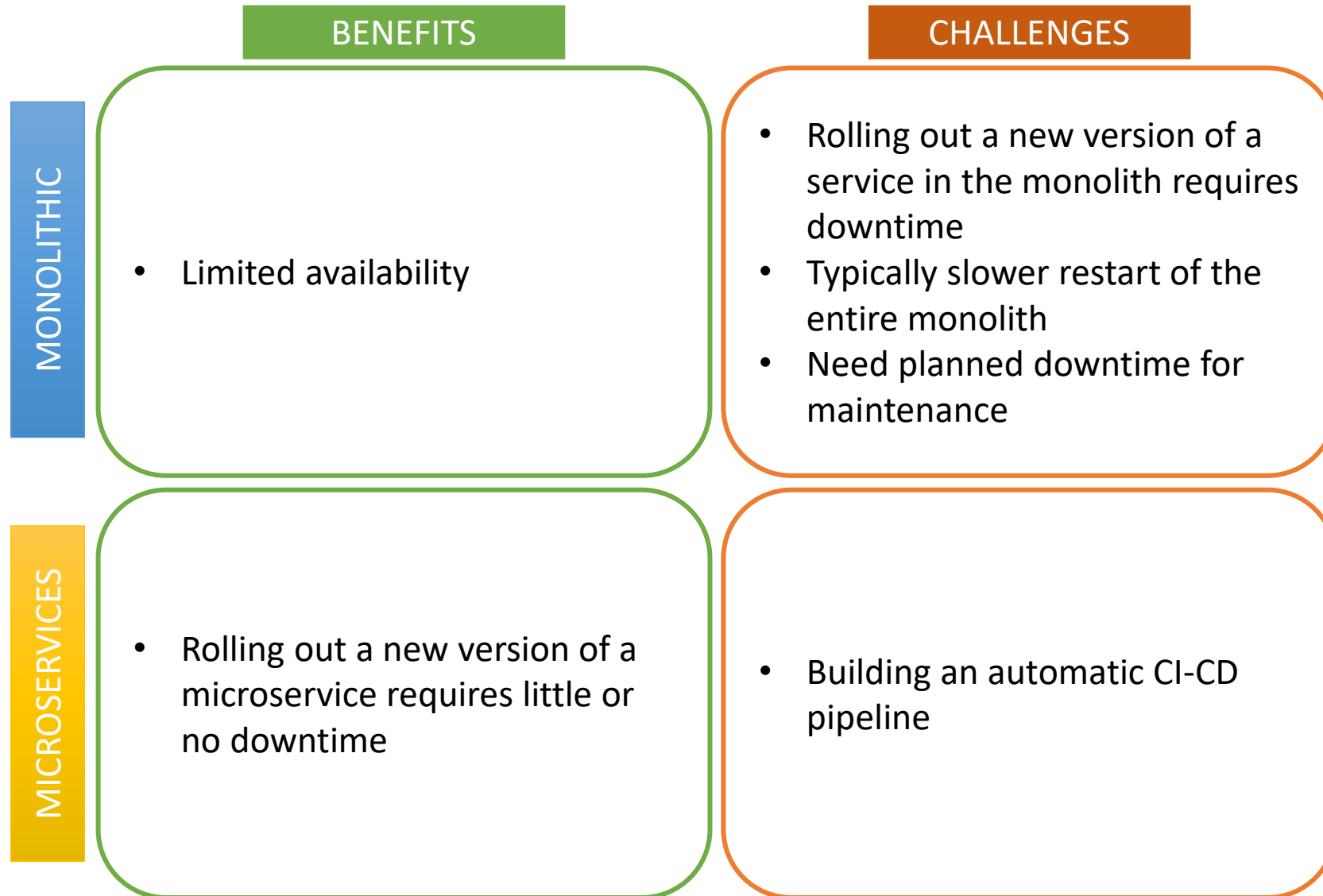
Performance



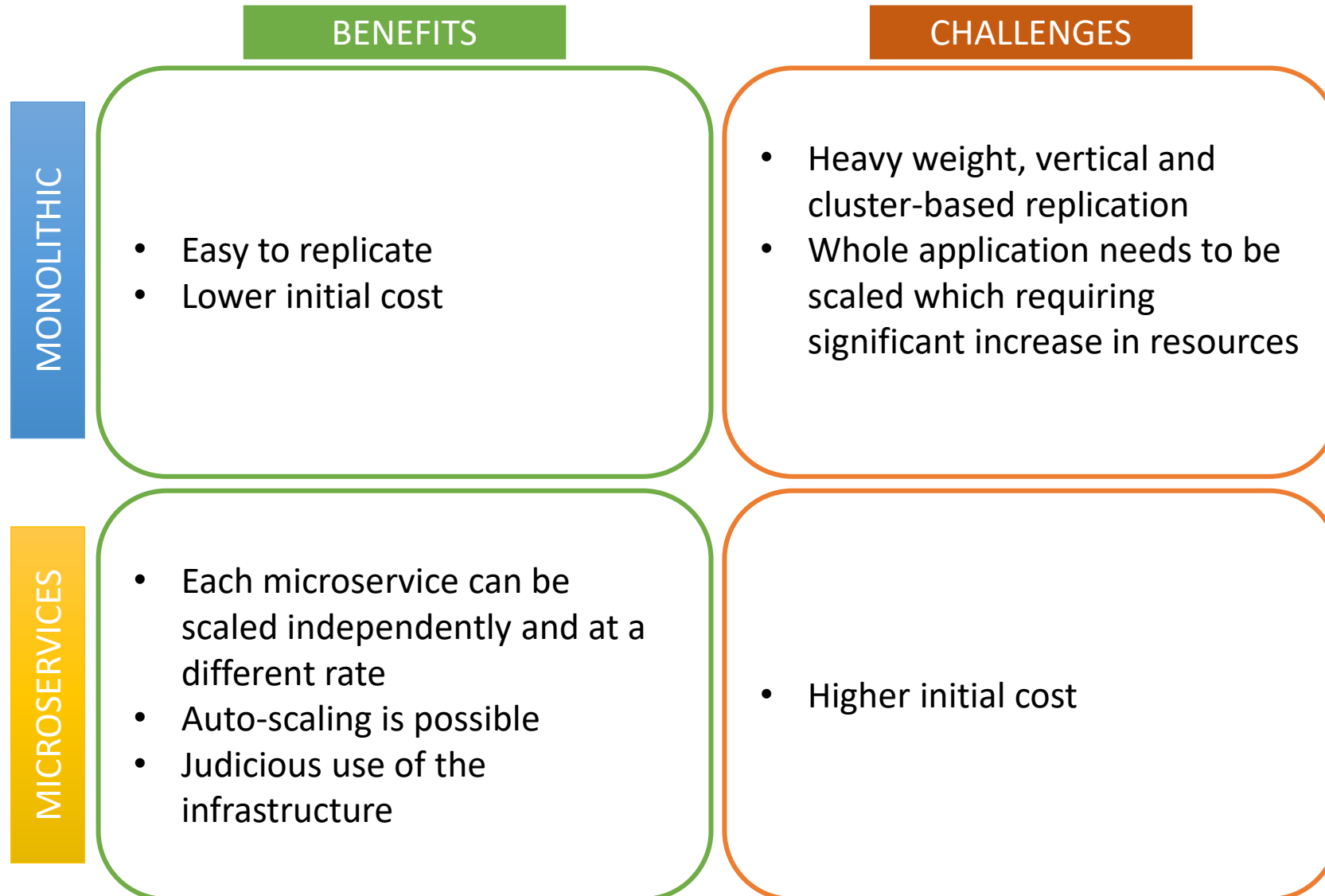
Reliability



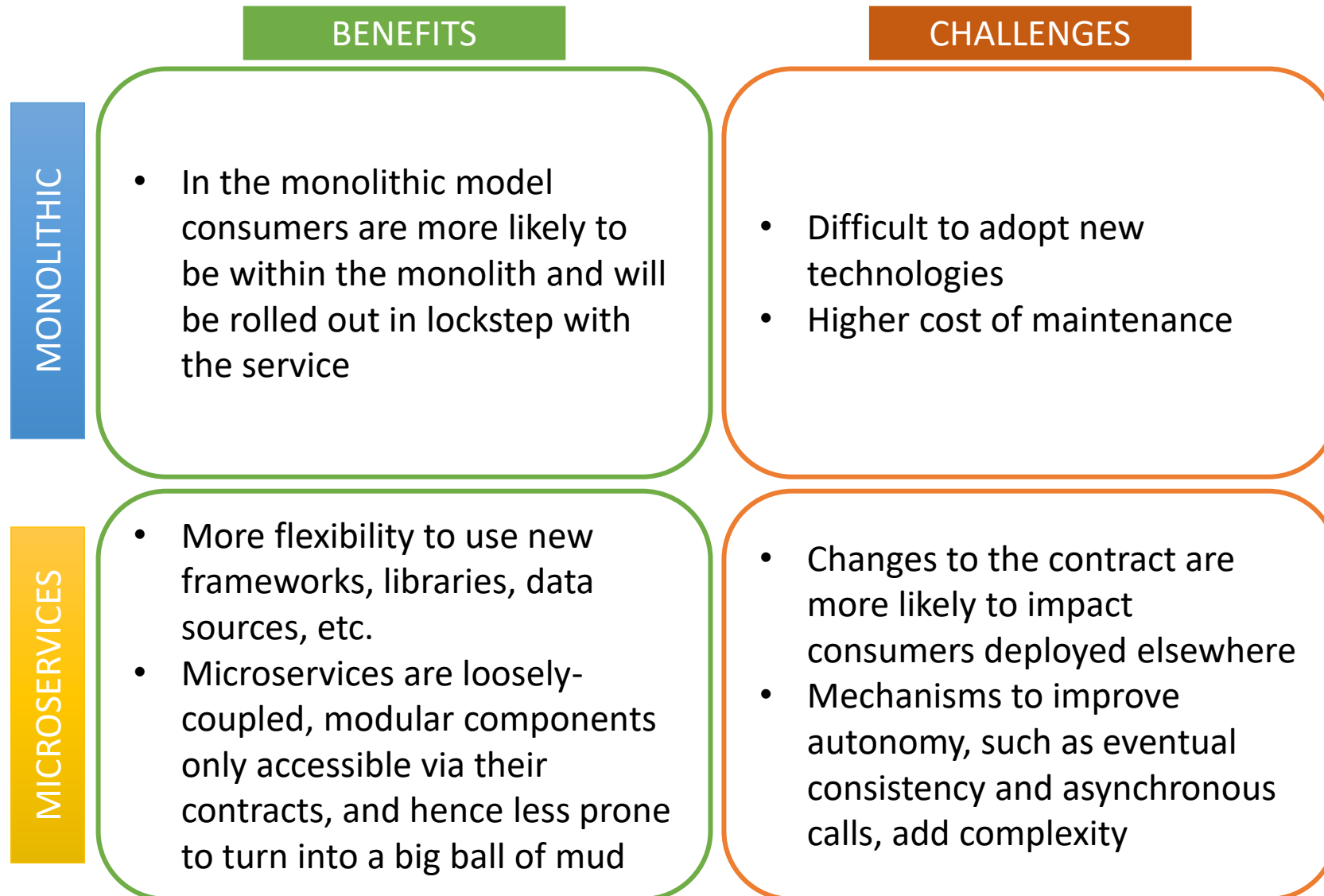
Availability



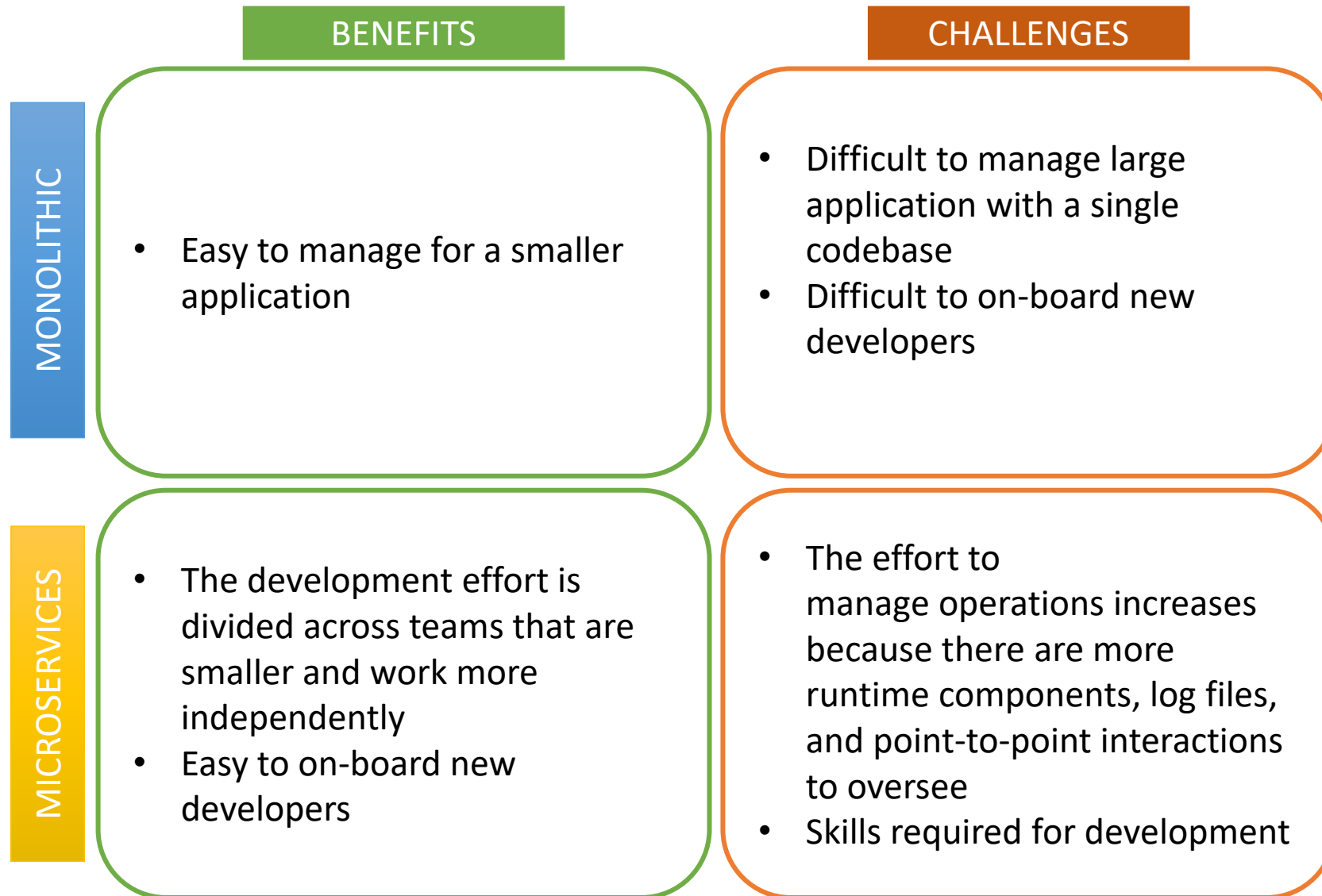
Scalability



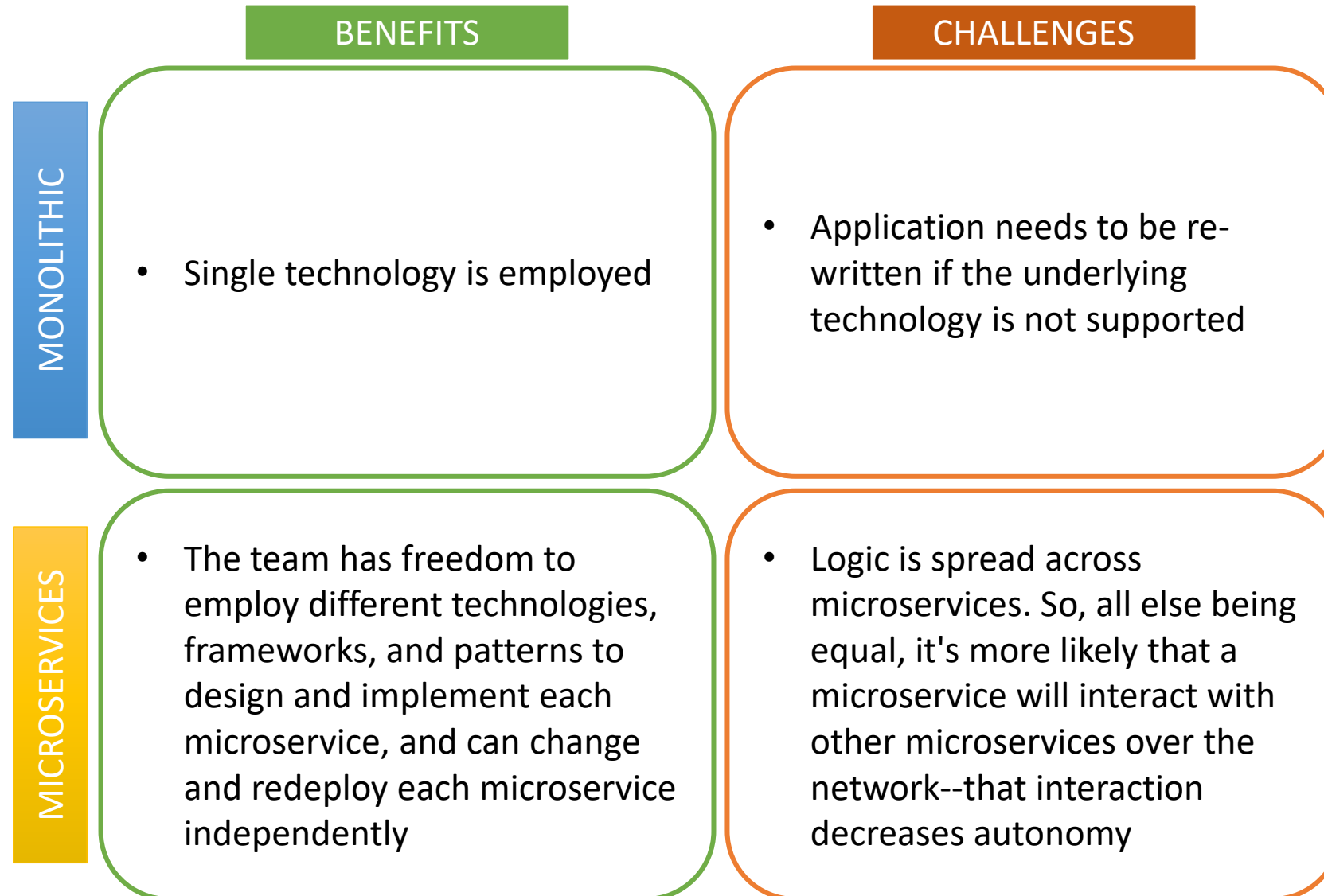
Modifiability



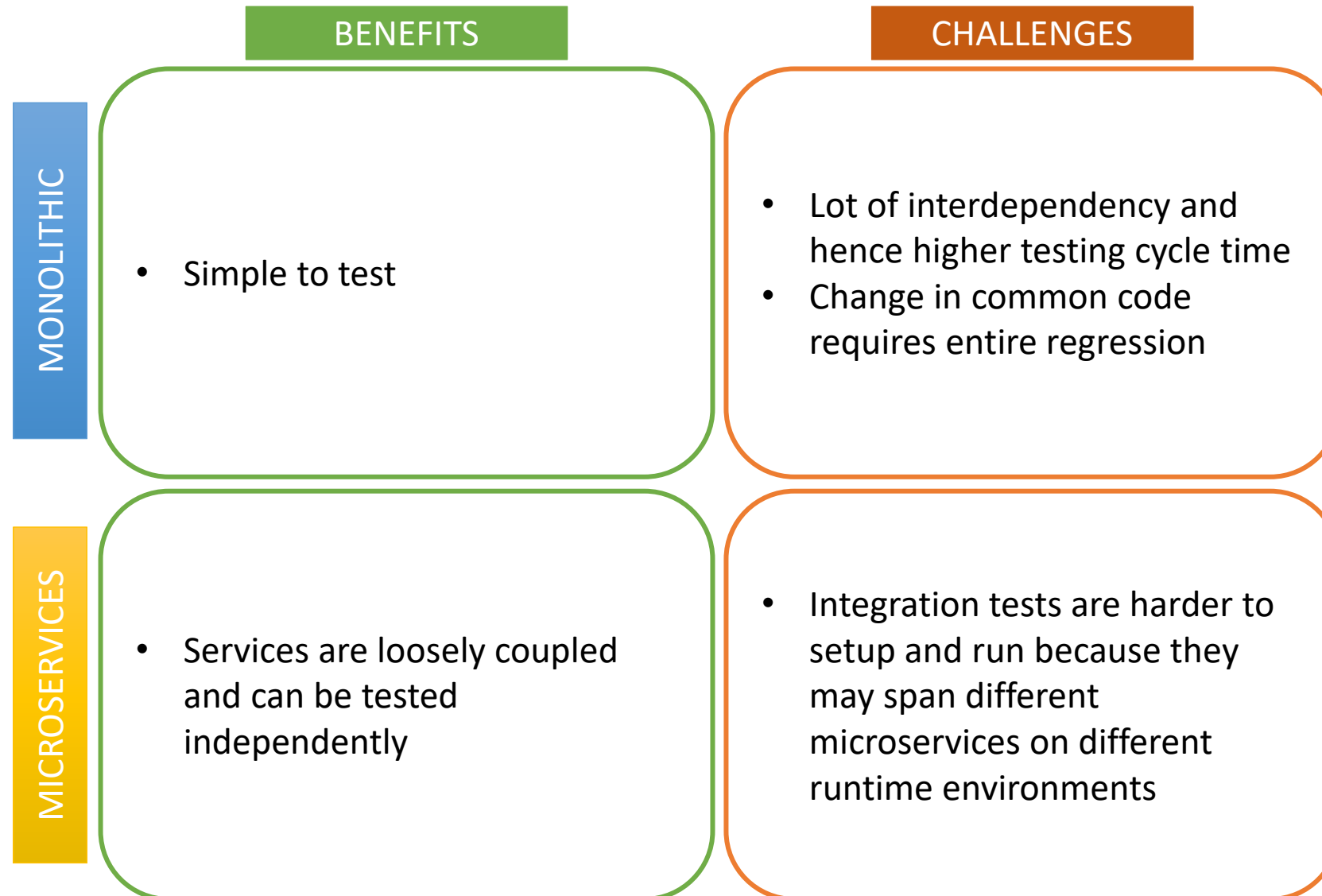
Management



Design Autonomy



Testability



Modular Monolithic – Game changer

- Modularizing Monolithic
 - > Fault tolerance
 - > Issue Isolation
 - > Componentized deployment
 - > Design simplicity
- Blue Green deployment
 - > Higher availability

Appendix – Architectural styles

Monolithic Architecture (MA)



Features

- UI, Business logic and data access combined in one application
- Single technology stack

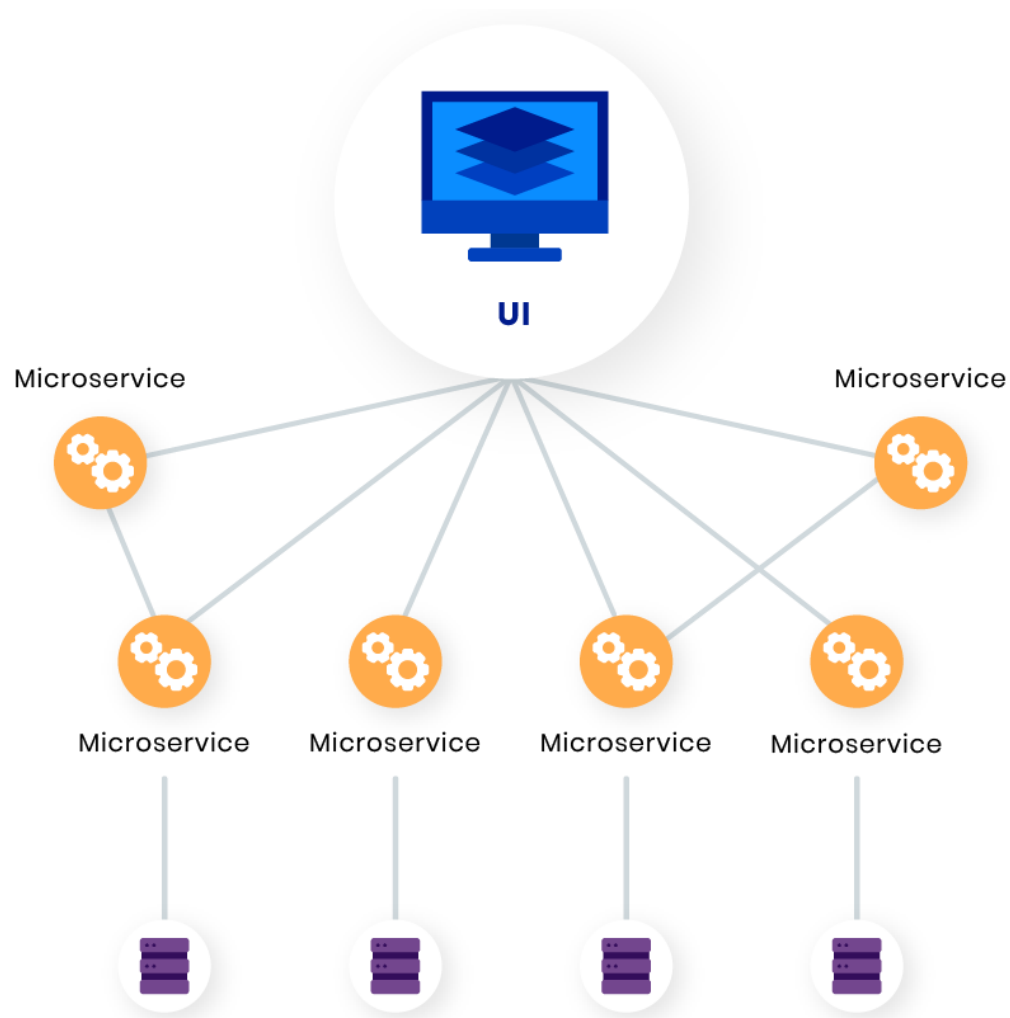
Pros

- Simple development
- Simple deployment and testing
- Single language and database

Cons

- Strong coupling and low cohesion
- Limited agility
- Barrier to new technologies
- Poor reliability
- Scalability

Micro Services Architecture (MSA)



Features

- Domain driven design (DDD)
- Highly cohesive and loosely coupled services
- Independent services with individual databases

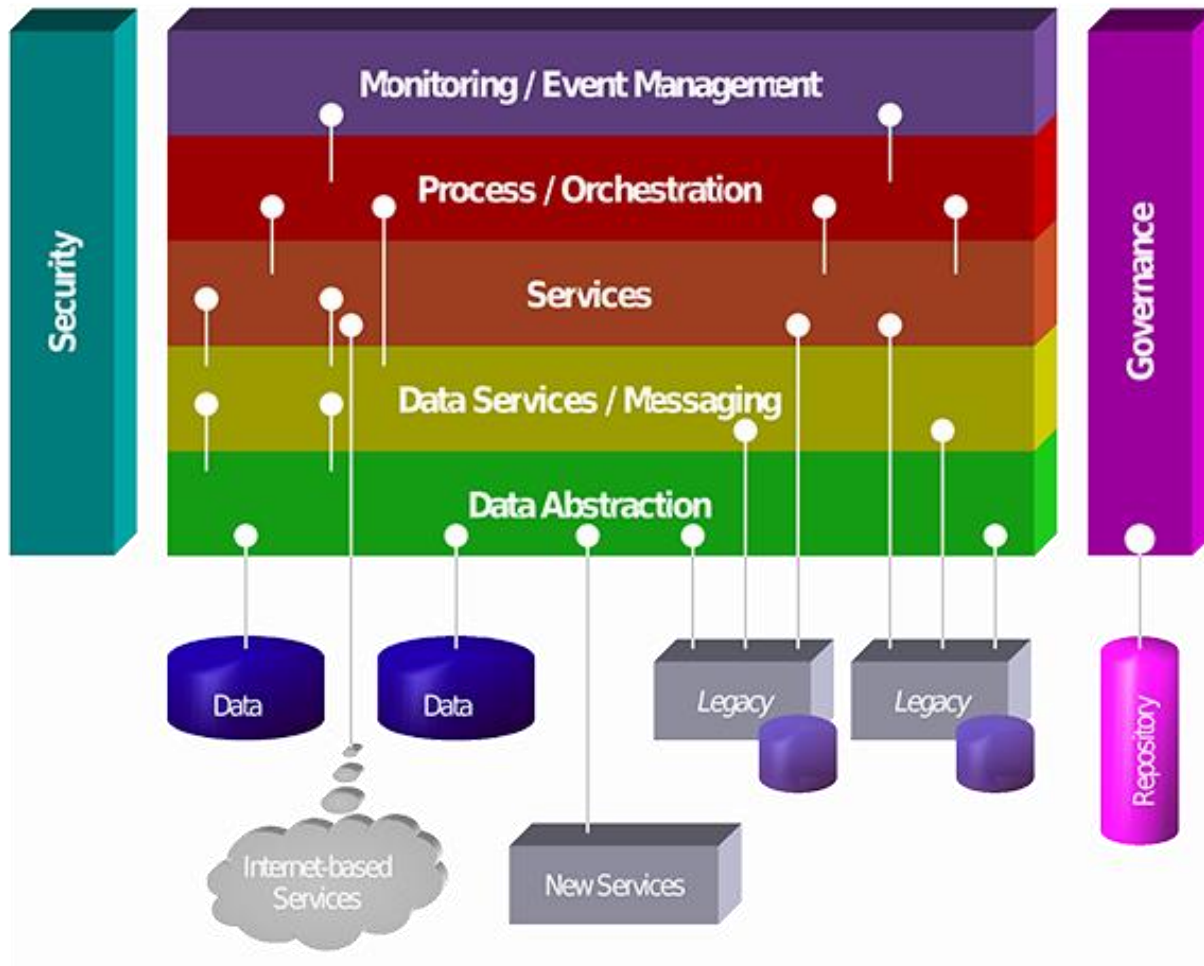
Pros

- Agility with CI-CD, DevOps
- Availability and Reliability
- Dynamic Scalability
- Maintainability

Cons

- Design and operational complexity
- Difficult Integration testing
- Time consuming
- Skills demand

Service Oriented Architecture (SOA)



Features

- Enterprise wide integration of business apps and processes
- Self contained shared Services
- Messaging using heavy weight ESB
- Model driven architecture

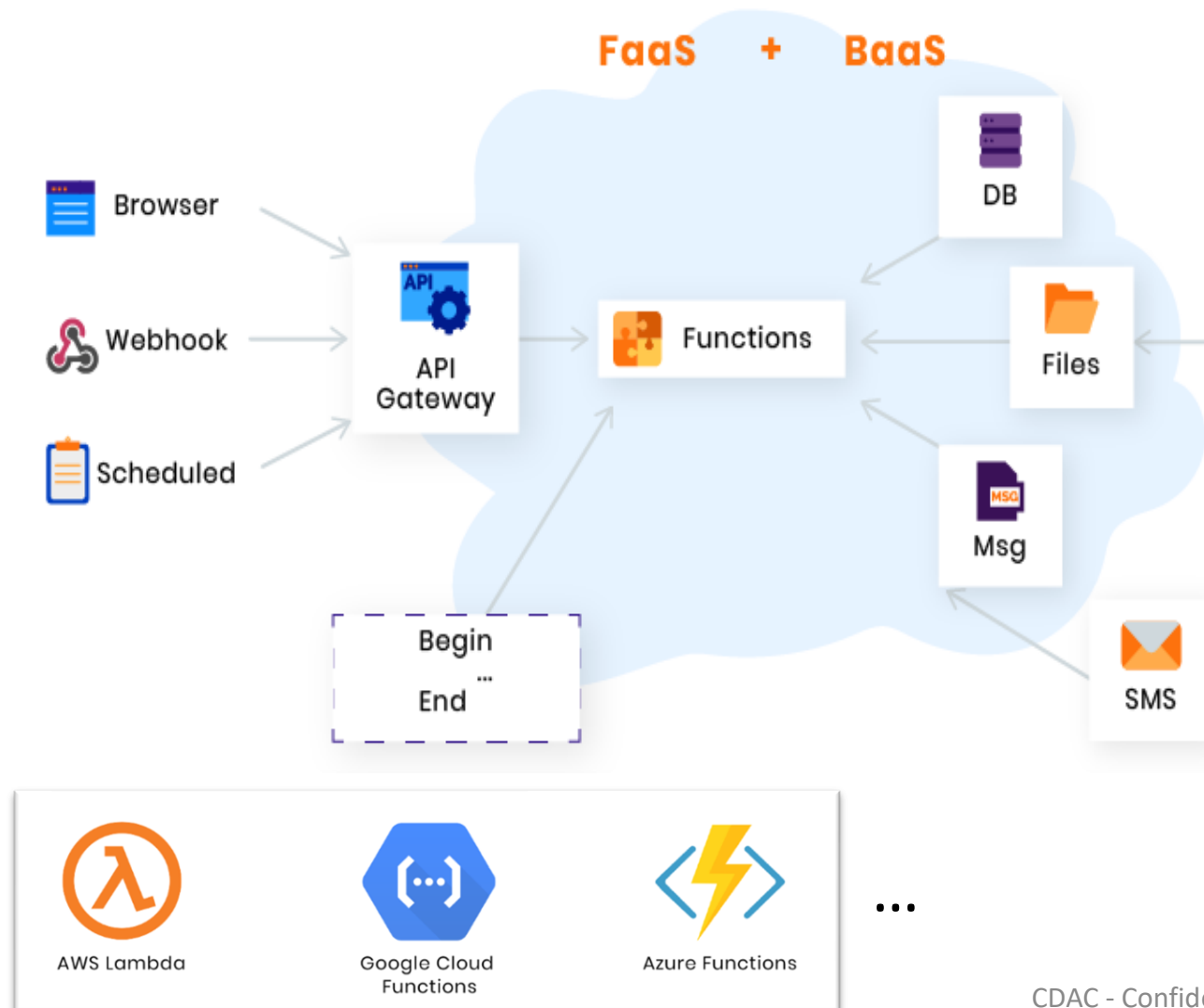
Pros

- Reusability of services
- Better maintainability
- Parallel development

Cons

- Complex service management
- Significant upfront investment
- Extra overload with increased response time and lower performance

Serverless Architecture (SA)



Features

- Cloud computing approach to building and running apps and services without the need of infrastructural management
- Third party cloud service takes care of the servers
- Incorporates FaaS and BaaS

Pros

- Easy to deploy
- Lower Costs
- Enhanced Scalability

Cons

- Vendor lock-in
- Not for long-term tasks

Thank you !